

PROYECTO-RETO

Programación y Entornos de Desarrollo

DAM1

Alejandro Díaz y Laura Leciñena Andres

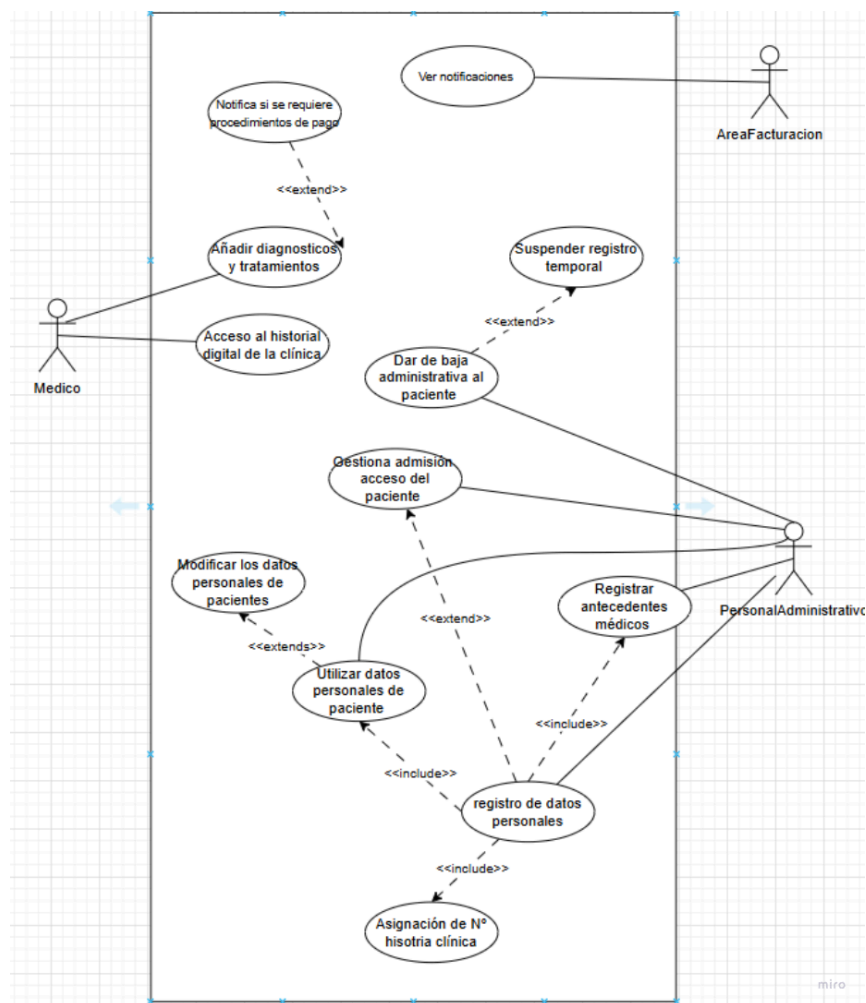
ÍNDICE

Diagrama de Casos de Uso	3
Fichas Diagrama de Casos de Uso	8
Diagrama de Clases	20
Diagrama Antiguo	20
Diagrama Actual	21
Clase Persona	23
Clase Medico	24
Clase PersonalAdmin	26
Clase Paciente	27
Clase AreaFacturacion	30
Clase HistorialMedico	33
Clase EstadoPacienteException	34
Diagramas de Comportamiento	35
Diagrama de Actividad:	35
Diagrama de Estados:	41
Diagrama de Estados Previo	41
Diagrama de Estados Actual	42
PLANTUML	46
Diagrama de caso de usos	46
Diagrama de actividad	46
Diagrama antiguo	46
Diagrama nuevo	47
Diagrama de estados	51
Diagrama antiguo	51
Diagrama nuevo	52
Diagrama de clases	57
Diagrama antiguo	57
Diagrama nuevo	61
Conversaciones	66
Reparto de Tareas	77

Diagrama de Casos de Uso

El diagrama de Casos de uso lo realizamos desde Draw.io , era uno de los programas donde era más completo para el diseño de diagramas de este tipo, ya que incluyen todos los iconos de PlantUML, nos permite realizar el diseño del diagrama de una forma mucho más cuidada y más estética visualmente.

Esta fue la primera propuesta de diagrama de casos de usos que hicimos, después de estar discutiendo con el profesor a diferencia del resultado final, hubieron varios cambios.



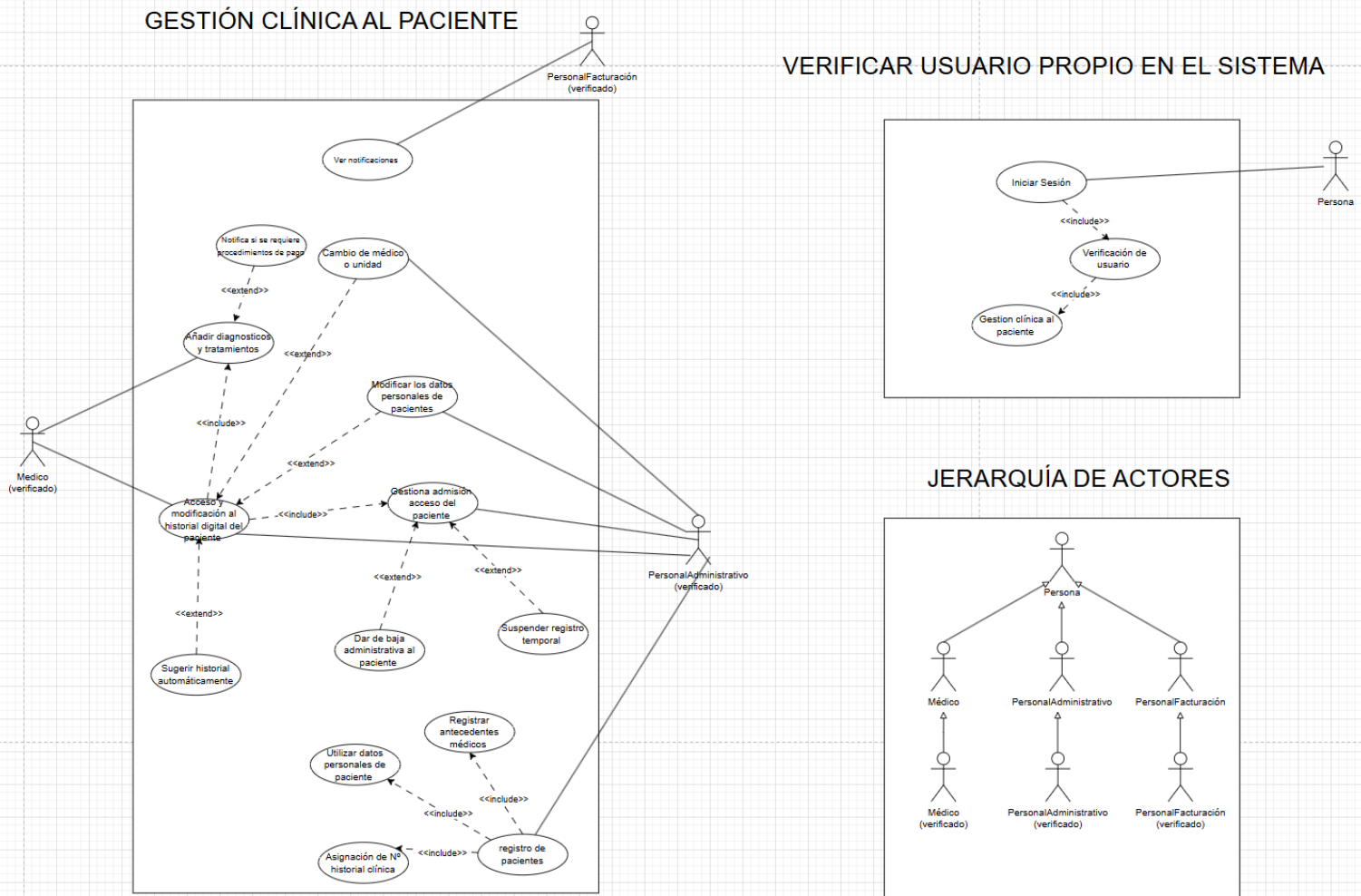
El principal problema era que no había forma de poder verificar a cada uno de los actores que interactúan con el sistema, así que decidimos que lo que se ve en la primera imagen a excepción de algunas cosas que se cambiarán posteriormente será el subsistema, y por encima antes de dirigirnos a él debemos de pasar por un sistema de verificación.

Para identificarlo lo que hemos hecho es para diferenciar los actores es colocar aquellos que se hayan podido verificar con éxito colocar por debajo (verificado)

También había error con la dirección de las flechas en el <<extend>> así que solo decidimos modificarlas con la orientación correcta.

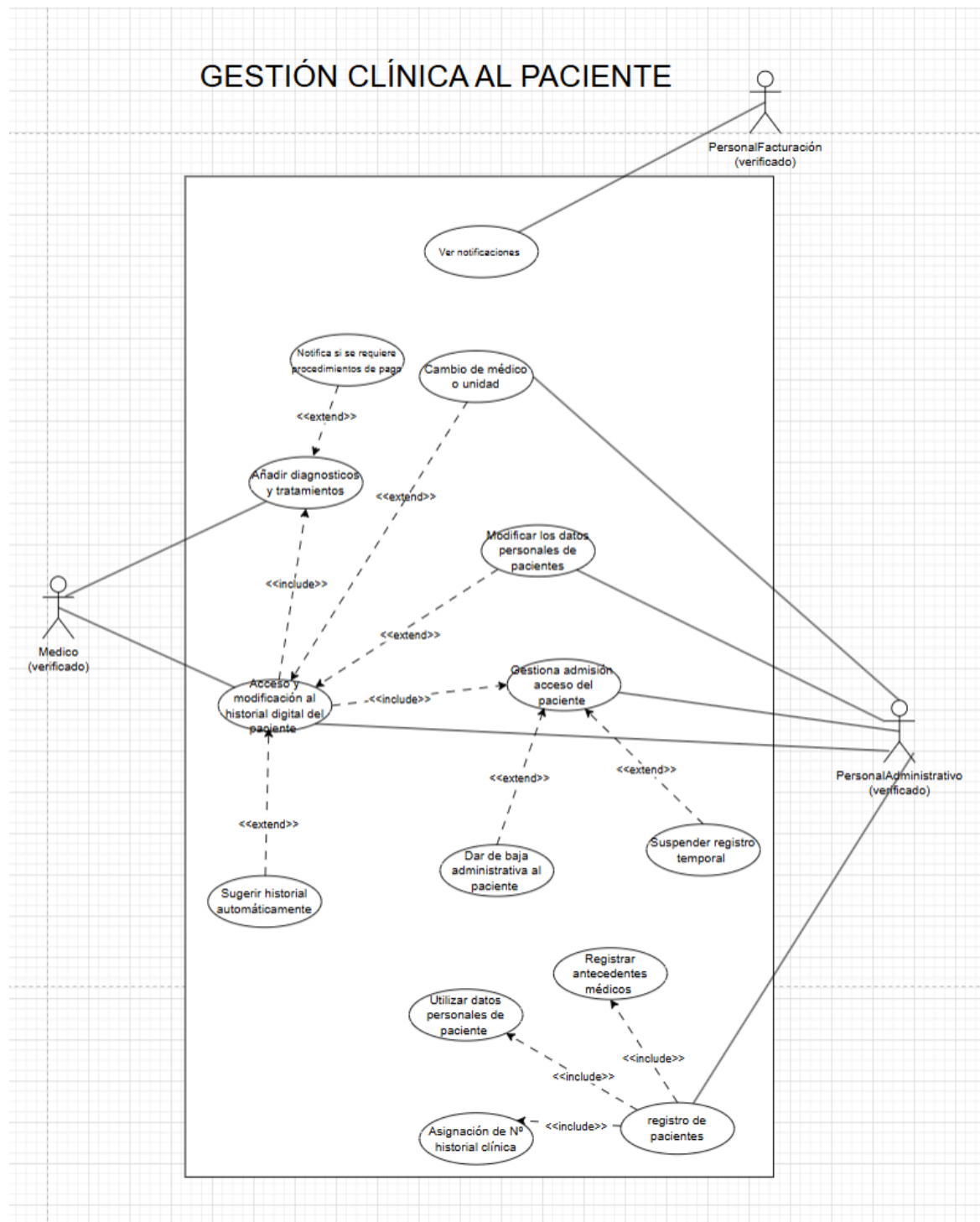
Proyecto-Reto: Programación y Entornos de Desarrollo

Otro problema era que no habíamos realizado una jerarquía de actores, para que de esa manera pudiéramos tener representado la clasificación de los actores y dejar mucho más claro de donde provienen cada uno de ellos.



La secuencia del primer diagrama es muy parecida a el resultado final con la diferencia de que este diagrama está mejor clasificado que el anterior.

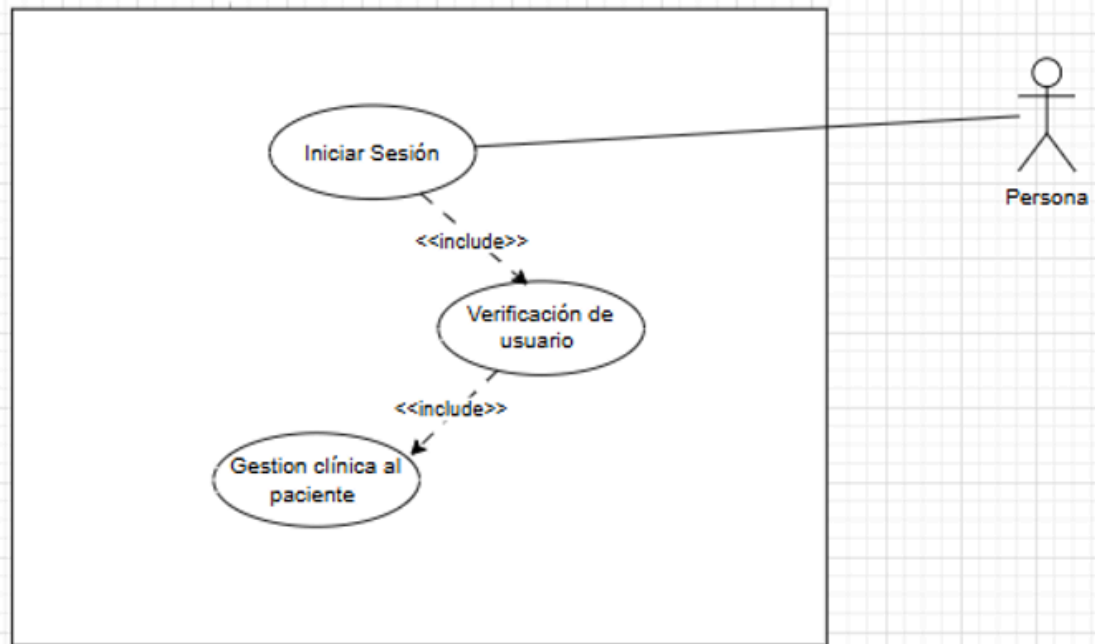
Todo fue comentado entre los integrantes del grupo y discutido, sobre todo algunos casos de usos, los cuales no se mencionan en el problema planteado pero sí que son necesarios para el correcto funcionamiento del sistema.



En el primer diagrama, se puede apreciar cómo hemos escogido solo 3 actores, Medico, PersonalFacturación, PersonalAdministrativo, cada uno cumple un rol y realiza funciones totalmente distintas dentro de nuestro programa, sin embargo interactúan dentro de él,

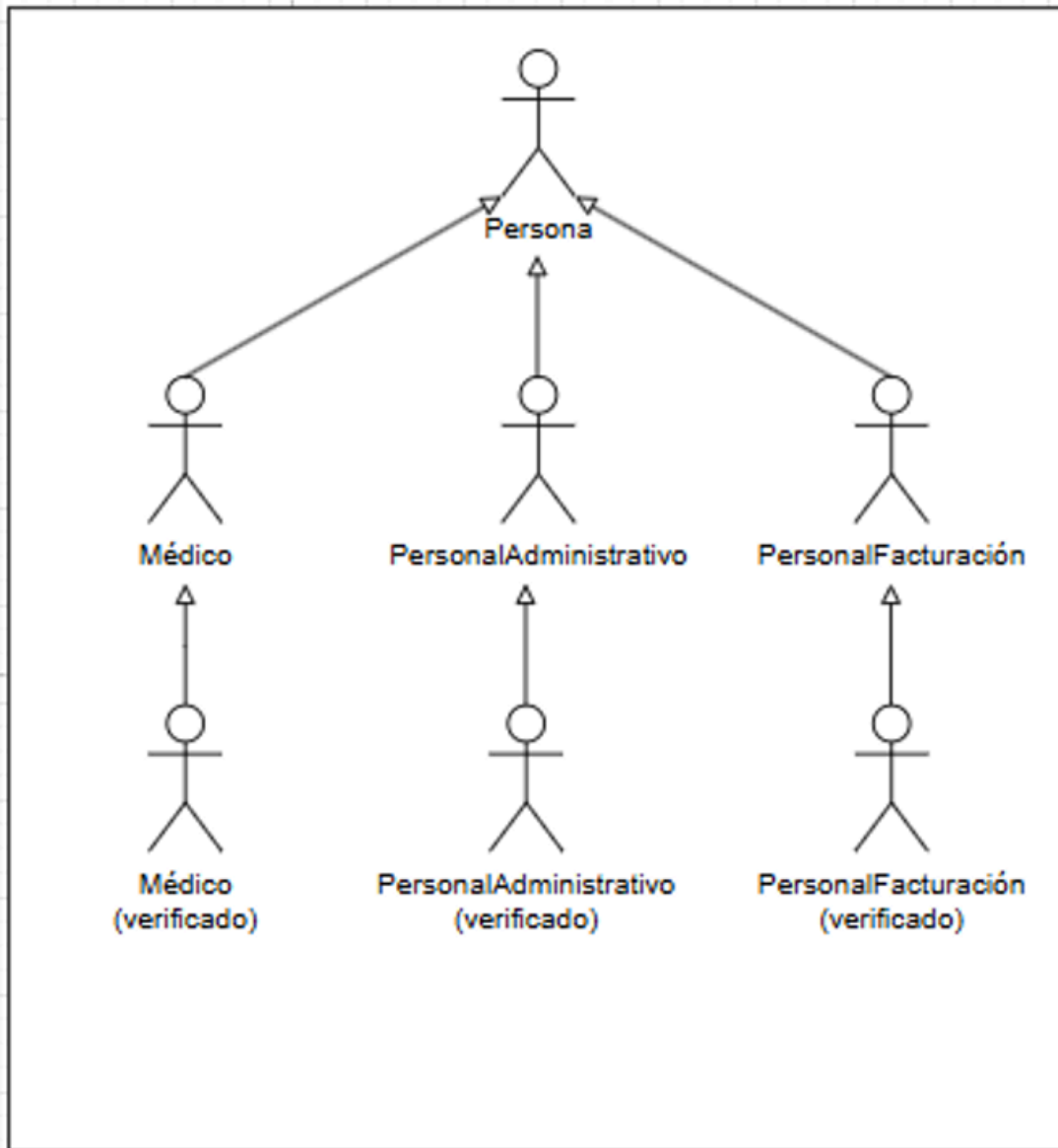
Proyecto-Reto: Programación y Entornos de Desarrollo

diferencia del paciente, ya que este no es una entidad que interactúa directamente con él tal y como hemos comentado anteriormente.



El diagrama Verificar usuario propio en el sistema es sencillo pero importante, en este diagrama cualquier persona que sea uno de los actores anteriormente mencionado (médico, personal administrativo, área facturación), podría interactuar con él, sin embargo no podrían iniciar sesión al menos que tengan un login previamente registrado, en caso de que si lo tenga y logre acceder de forma efectiva le redirigirá a la gestión clínica al paciente y en caso de que no tendrá que volver a intentarlo.

JERARQUÍA DE ACTORES



Esta jerarquía explica de forma más específica lo que mencionamos anteriormente, al sistema solo pueden acceder 3 tipos de personas, lo cuales antes de haber iniciado sesión aún no se encuentra por razones obvias logueadas, una vez que pase el proceso de **Verificación de usuario propio en el sistema**, aparecerán como verificados y únicamente si lo están podrán acceder el sistemas y utilizar sus funciones.

Fichas Diagrama de Casos de Uso

Nombre:	Ver notificaciones
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El personal del Área de Facturación recibe una notificación de requerimiento de pago .	
Actores: PersonalFacturacion	
Precondiciones: 1. El médico haya enviado la notificación a la bandeja de notificaciones del sistema del Personal de Facturación.	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Notifica si se requiere procedimientos de pago> 2. El personal revisa la bandeja de notificaciones dentro del sistema. 3. El personal selecciona el tratamiento del respectivo paciente el cual se cobrará. 4. El personal cobra al paciente por el tratamiento y diagnóstico.	
Flujo Alternativo: 2a. El personal al revisar la bandeja de notificaciones dentro del sistema no encuentra ningún procedimiento de pago pendiente.	
Poscondiciones: El paciente recibirá el cobro o no dependiendo de si el médico considera que debería de enviarse.	

Nombre:	Notifica si se requiere procedimiento de pago
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El médico realiza una notificación sobre el requerimiento del pago debido al tratamiento dado al paciente	
Actores: Actor Primario: Medico Actor secundario: PersonalFacturacion	
Precondiciones: 1. El médico considera que se debe realizar un cobro al paciente debido al tratamiento el	

Proyecto-Reto: Programación y Entornos de Desarrollo

cual se le ha realizado.
Flujo Normal: 1. El sistema ejecuta el caso de uso <Añadir diagnósticos y tratamientos> 2. El médico evalúa si se requiere o no procedimiento de pago. 3. El médico envía la notificación de cobro al respectivo paciente a la bandeja del Personal de Facturación para realizar el cobro correspondiente
Flujo Alternativo: 2a. El médico decide que el tratamiento o el diagnóstico realizado al paciente no es lo suficiente como para realizar un cobro. 2a1. No envía la notificación al Personal Administrativo
Poscondiciones: El médico enviará o no la notificación dependiendo de que considere si se deba realizar el cobro al paciente

Nombre:	Añadir Diagnósticos y tratamientos
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El médico añade diagnósticos o tratamientos dependiendo de la situación o necesidad que tenga cada paciente, modificando su historial médico.	
Actores: Medico	
Precondiciones: 1. Haber accedido correctamente al historial médico del paciente al cual se tratará	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Acceso y modificación al historial digital del paciente> 2. Realiza el médico la consulta completa con el paciente 3. Añade diagnóstico y tratamientos al historial médico. 4. Guarda los datos en el historial médico del paciente	
Flujo Alternativo: 2a. Que el paciente no asista a la consulta 4a. Ocurre un error al querer añadir información al historial médico	
Poscondiciones: El médico añadirá diagnósticos y tratamientos si realiza de forma efectiva la cita con el médico	

Proyecto-Reto: Programación y Entornos de Desarrollo

Nombre:	Acceso y modificación al historial digital del paciente
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Realiza el acceso al historial médico del paciente y también permite la modificación del mismo para poder añadir información correspondiente a las consultas que se realicen. En este caso de uso pueden acceder los actores Medico y PersonalAdministrativo	
Actores: Medico, PersonalAdministrativo	
Precondiciones: 1. Haber podido validar la existencia del historial del paciente correctamente.	
Flujo Normal: 1. Los actores introducen el número del Historial Clínico del paciente y el DNI 2. Valida si el dato existe dentro del sistema de gestión 3. Accede al historial médico del paciente 4. Muestra todos los datos personales del paciente y también las antiguas consultas.	
Flujo Alternativo: 2a. No existe el número del Historial Clínico y no está vinculado el DNI del paciente 2a1. Vuelve al punto 1	
Poscondiciones: Entrará o no dependiendo de si su número de Historial Clínico existe dentro de la base de datos del sistema de Gestión del paciente.	

Nombre:	Sugerir historial automáticamente
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Este caso de uso realiza la funcionalidad de mostrar de forma automática el historial médico de aquellos pacientes que sean recurrentes para que de esa forma sea más fácil acceder a sus datos más fácil	
Actores: Medico, PersonalAdministrativo	

Proyecto-Reto: Programación y Entornos de Desarrollo

Precondiciones: 1. Que el paciente sea recurrente dentro del centro clínico.
Flujo Normal: 1. Cualquiera de los dos actores que interactúan con el caso de uso buscan al paciente mediante el número de historial clínico 2. Validamos si se encuentra dentro de la base de datos de la tabla pacientes. 3. Accedemos de forma automática a todos sus datos de forma directa.
Flujo Alternativo: 2a. No se ha encontrado el número del historial clínico en la base de datos 2a1. Vuelve al punto 1
Poscondiciones: Entrará o no dependiendo de si el número del historial médico se encuentra en el sistema

Nombre:	Registro de pacientes
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Registra a los nuevos pacientes, generando el alta administrativa, para poder pasar consulta dentro del centro clínico	
Actores: PersonalAdministrativo	
Precondiciones: 1. No estar registrad@ con anterioridad en el centro clínico.	
Flujo Normal: 1. El paciente pide al personal administrativo el alta administrativo para poder pasar consulta dentro del centro clínico 2. El personal Administrativo solicita el caso de uso <Utilizar datos personales de paciente> 3. El personal Administrativo solicita el caso de uso <Registrar antecedentes médicos> 4. Al completar el registro nos devuelve el caso de uso <Asignación de N° historial clínica>	
Flujo Alternativo: 1a. El paciente no pueda registrarse ya que se encontraba en el sistema con anterioridad	
Poscondiciones: El paciente podrá registrarse únicamente si no lo ha realizado anteriormente.	

Nombre:	Registrar antecedentes médicos
----------------	--------------------------------

Proyecto-Reto: Programación y Entornos de Desarrollo

Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Integra dentro del historial médico los antecedentes que tiene el paciente de otros centros clínicos	
Actores: PersonalAdministrativo	
Precondiciones: 1. Haber solicitado los antecedentes médicos del centro clínico del que proviene.	
Flujo Normal: 1. Ejecutar el caso de uso <Utilizar datos personales de pacientes> 2. Dentro del registro del paciente debemos de añadir los antecedentes médicos que proviene del centro clínico anterior 3. Guardamos esa información en el historial clínico de nuestro sistema	
Flujo Alternativo: 3a. No se guardará la información en caso de que no tenga antecedentes médicos provenientes de otro centro clínico	
Poscondiciones: Se almacenarán los antecedentes médicos de otro centro clínico dentro del historial únicamente si estos existen.	

Nombre:	Utilizar datos personales de paciente
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Para completar el registro se solicitarán los datos personales del paciente y también los datos de contacto, para completar la información del historial clínico dentro del sistema	
Actores: PersonalAdministrativo	
Precondiciones: Tener a disposición todos los datos los cuales se van a solicitar para efectuar correctamente el registro del paciente en el sistema.	
Flujo Normal: 1. Se ejecuta el caso de uso <registro de pacientes> 2. El personal Administrativo le solicita al paciente el Nombre, dni, número de teléfono, número de emergencia, dirección, fecha de nacimiento, sexo y correo electrónico. 3. El personal Administrativo crea su nueva ficha dentro del sistema e introduce los nuevos	

Proyecto-Reto: Programación y Entornos de Desarrollo

datos 4. Los datos son almacenados en la base de datos.
Flujo Alternativo: 2a. Si el paciente no posee los datos los cuales se le solicitarán para realizar adecuadamente el registro, no se podrá efectuar su incorporación dentro del sistema
Poscondiciones: Si el paciente posee todos los datos necesarios para realizar el registro correctamente se podrá efectuar de forma efectiva

Nombre:	Asignación de N° historial clínica
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Una vez completado el registro de manera efectiva, se genera un número de historia clínica única, totalmente personal para cada uno de los pacientes que se encuentran registrados dentro del sistema	
Actores: Personal/Administrativo	
Precondiciones: 1. Haber realizado el caso de uso de manera correcta <Registrar antecedentes médicos> 2. haber realizado el caso de uso de manera correcta <Utilizar datos personales de paciente> 3. Haber realizado el registro correctamente	
Flujo Normal: 1. Se ejecuta el caso de uso <Registrar antecedentes médicos> 2. Una vez completado, realizaremos el registro dentro de nuestro sistema 3. Nos devolverá el número de historial clínica respectivo al paciente que acaba de ser registrado en nuestro sistema de forma automática.	
Flujo Alternativo: 2a. Si no se realiza el registro no podrá generarse el número de historial clínica	
Poscondiciones: Si se ha realizado correctamente el registro generará de forma automática el número del historial clínico del paciente.	

Nombre:	Gestión admisión acceso del paciente
----------------	--------------------------------------

Proyecto-Reto: Programación y Entornos de Desarrollo

Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El personal Administrativo controla la admisión del paciente, maneja el flujo de acceso desde su ingreso hasta su salida	
Actores: PersonalAdministrativo	
Precondiciones: 1. El paciente debe estar registrado en el sistema. 2. El Personal Administrativo debe haber iniciado sesión correctamente en el sistema	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Acceso y modificación al historial digital del paciente>. 2. El Personal Administrativo verifica la llegada del paciente al hospital y lo registra en el sistema 3. Se le asigna una consulta con su respectivo médico 4. El médico atiende al paciente. 5. Al finalizar la atención, el Personal Administrativo registra que ha concluido la visita del paciente en el sistema.	
Flujo Alternativo: 2a. El paciente no se presenta a su cita. 2a1. El Personal Administrativo registra la ausencia del paciente en el sistema 2a2. El caso de uso termina sin continuar el flujo de atención 5a. El paciente requiere atención adicional o nuevas gestiones. 5a1. El flujo continúa con las gestiones necesarias antes de cerrar el registro de atención	
Poscondiciones: El sistema refleja el ingreso y la salida del paciente correctamente en el historial de admisiones	

Nombre:	Modificar los datos personales de pacientes
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El Personal Administrativo realiza modificaciones sobre los datos personales de un paciente ya registrado en el sistema para mantener la información actualizada y precisa	
Actores: PersonalAdministrativo	
Precondiciones: 1. El paciente debe estar previamente registrado en el sistema. 2. Debe existir una necesidad de actualización o corrección en los datos del paciente	

Proyecto-Reto: Programación y Entornos de Desarrollo

Flujo Normal:

1. El sistema ejecuta el caso de uso **<Acceso y modificación al historial digital de paciente>**.
2. El Personal Administrativo accede al registro del paciente mediante su número de historial clínico o DNI (depende si es un paciente recurrente)
3. El sistema muestra los datos actuales del paciente
4. El Personal Administrativo modifica los datos que requieran actualización (dirección, número de teléfono, correo electrónico, teléfono de emergencia, etc.)
5. El sistema almacena los cambios realizados en la base de datos

Flujo Alternativo:

2a. El número de historial clínico o DNI introducido no existe en el sistema.

2a1. El sistema muestra un mensaje de error indicando que el paciente no se encuentra registrado

2a2. El caso de uso vuelve al paso 2.

5a. Ocurre un error al intentar guardar los cambios en el sistema.

5a1. El sistema muestra un mensaje de error y los cambios no se almacenan

5a2. El caso de uso vuelve al paso 4 para que el Personal Administrativo intente guardar de nuevo

Poscondiciones:

Los datos personales del paciente quedan actualizados correctamente en el sistema.

Flujo alternativo: Los datos no se modifican si el paciente no existe o si ocurre un error al guardar

Nombre:	Cambio de médico o unidad
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El Personal Administrativo gestiona la solicitud de cambio de médico o unidad de un paciente, actualizando su historial médico para reflejar el nuevo médico o unidad asignada.	
Actores: PersonalAdministrativo	
Precondiciones: 1. El paciente debe estar registrado y activo en el sistema 2. El paciente debe haber solicitado el cambio de médico o unidad	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Acceso y modificación al historial digital de paciente>.	

Proyecto-Reto: Programación y Entornos de Desarrollo

2. El Personal Administrativo accede al registro del paciente
3. El Personal Administrativo registra el nuevo médico o unidad asignada al paciente
4. El sistema actualiza el historial médico del paciente con el nuevo médico o unidad.
5. El sistema almacena los cambios en la base de datos

Flujo Alternativo:

- 3a. El médico o la unidad solicitada no se encuentra disponible en el sistema
- 3a1. El sistema informa al Personal Administrativo de la falta de disponibilidad
- 3a2. El Personal Administrativo busca una alternativa disponible
- 3a3. El caso de uso vuelve al paso 3.

Poscondiciones:

El historial clínico del paciente queda actualizado con el nuevo médico o unidad asignada.
Flujo alternativo: Si no hay disponibilidad, no se realiza ningún cambio hasta encontrar una alternativa.

Nombre:	Suspender registro temporal
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El Personal Administrativo realiza la suspensión temporal del registro de un paciente en el sistema, inhabilitándolo de forma provisional sin eliminarlo definitivamente de la base de datos	
Actores: PersonalAdministrativo	
Precondiciones: 1. El paciente debe estar registrado y activo en el sistema. 2. Debe existir un motivo justificado para la suspensión temporal del registro	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Gestión admisión acceso del paciente>. 2. El Personal Administrativo accede al registro del paciente 3. El Personal Administrativo selecciona la opción de suspensión temporal del registro 4. El sistema solicita confirmación de la acción 5. El Personal Administrativo confirma la suspensión 6. El sistema cambia el estado del paciente a BAJA_TEMPORAL y lo registra en el historial.	
Flujo Alternativo: 5a. El Personal Administrativo cancela la acción antes de confirmar 5a1. El sistema no realiza ningún cambio en el registro del paciente. 5a2. El caso de uso termina sin modificar el estado del paciente	
Poscondiciones: El registro del paciente queda suspendido temporalmente en el sistema con estado BAJA_TEMPORAL.	

Proyecto-Reto: Programación y Entornos de Desarrollo

Nombre:	Dar de baja administrativa al paciente
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El Personal Administrativo realiza la baja definitiva de un paciente en el sistema, eliminando o desactivando su registro de forma permanente según las políticas del hospital	
Actores: PersonalAdministrativo	
Precondiciones: 1. El paciente debe estar registrado en el sistema. 2. Debe existir una justificación válida para la baja definitiva	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Gestión admisión acceso del paciente>. 2. El Personal Administrativo accede al registro del paciente 3. El Personal Administrativo selecciona la opción de dar de baja definitiva al paciente 4. El sistema solicita confirmación de la acción 5. El Personal Administrativo confirma la baja 6. El sistema cambia el estado del paciente a BAJA_PERMANENTE y desactiva su registro. 7. El sistema cierra el registro de atención del paciente	
Flujo Alternativo: 5a. El Personal Administrativo cancela la acción antes de confirmar 5a1. El sistema no realiza ningún cambio en el registro del paciente 5a2. El caso de uso termina sin modificar el estado del paciente 6a. El sistema detecta que el paciente tiene facturas pendientes de cobro. 6a1. El sistema avisa al Personal Administrativo de las facturas pendientes antes de completar la baja. 6a2. El Personal Administrativo decide si continúa con la baja o la cancela	
Poscondiciones: El registro del paciente queda dado de baja de forma permanente con estado BAJA_PERMANENTE y su registro de atención queda cerrado	

Nombre:	Iniciar sesión
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El usuario del sistema (Médico, Personal Administrativo o Personal de Facturación) introduce sus credenciales para acceder al sistema de gestión clínica.	
Actores: Persona (Médico, PersonalAdministrativo, PersonalFacturacion)	
Precondiciones: 1. El usuario debe estar registrado en el sistema con credenciales válidas	

Proyecto-Reto: Programación y Entornos de Desarrollo

2. El sistema debe estar operativo.
Flujo Normal: 1. El usuario accede a la pantalla de inicio de sesión del sistema 2. El usuario introduce su nombre de usuario y contraseña 3. El sistema ejecuta el caso de uso <Verificar usuario>. 4. El sistema valida las credenciales introducidas. 5. El sistema redirige al usuario al sistema de gestión clínica correspondiente según su rol
Flujo Alternativo: 2a. El usuario no recuerda sus credenciales. 2a1. El usuario puede solicitar la recuperación de credenciales al Personal Administrativo 2a2. El caso de uso vuelve al paso 2 4a. Las credenciales introducidas son incorrectas 4a1. El sistema muestra un mensaje de error indicando que las credenciales no son válidas. 4a2. El caso de uso vuelve al paso 2
Poscondiciones: El usuario accede correctamente al sistema de gestión clínica con los permisos correspondientes a su rol.

Nombre:	Verificar usuario
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: El sistema comprueba la validez de las credenciales introducidas por el usuario y verifica que existe en la base de datos del sistema, determinando su rol y nivel de acceso	
Actores: Persona (Médico, PersonalAdministrativo, PersonalFacturacion)	
Precondiciones: 1. El usuario debe haber ejecutado el caso de uso <Iniciar sesión>. 2. El usuario debe haber introducido sus credenciales (nombre de usuario y contraseña)	
Flujo Normal: 1. El sistema recibe las credenciales introducidas por el usuario 2. El sistema consulta la base de datos para comprobar si el usuario existe 3. El sistema verifica que la contraseña introducida coincide con la almacenada 4. El sistema determina el rol del usuario (Médico, PersonalAdministrativo, PersonalFacturacion) 5. El sistema marca al usuario como verificado y le redirige al sistema de gestión clínica al paciente	
Flujo Alternativo: 2a. El nombre de usuario no existe en la base de datos	

Proyecto-Reto: Programación y Entornos de Desarrollo

2a1. El sistema devuelve un error de credenciales inválidas al caso de uso <Iniciar sesión>.
2a2. El caso de uso termina sin conceder acceso.

3a. La contraseña introducida no coincide con la almacenada.

3a1. El sistema devuelve un error de credenciales inválidas al caso de uso <Iniciar sesión>.

3a2. El caso de uso termina sin conceder acceso.

Poscondiciones:

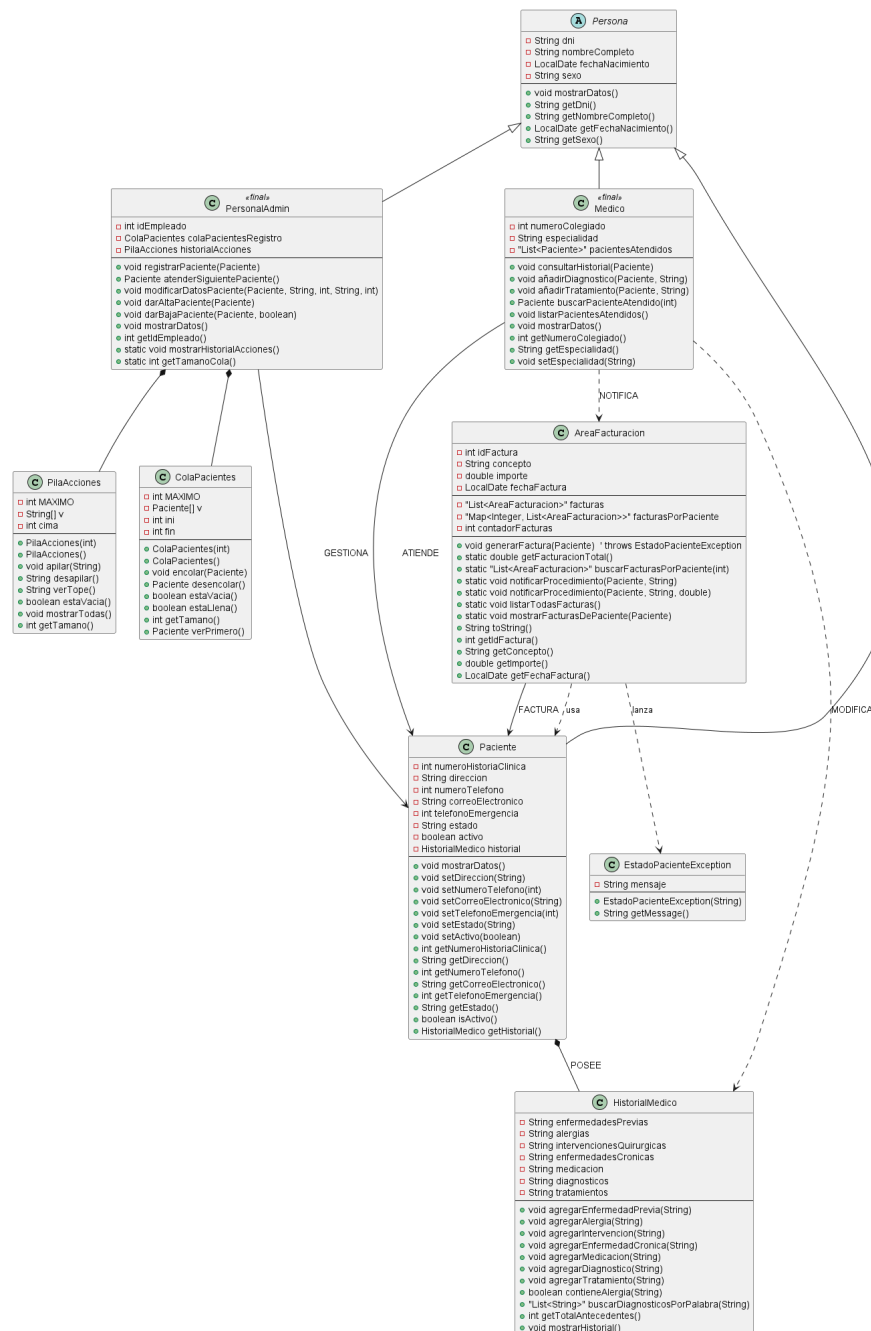
El usuario queda verificado en el sistema y puede acceder a las funcionalidades correspondientes a su rol.

Nombre:	Gestión clínica al paciente
Autor:	José Alejandro Díaz Delgado, Laura Leciñena Andres
Fecha:	04/05/2026
Descripción: Representa el subsistema principal de gestión clínica al paciente. Agrupa todas las funcionalidades relacionadas con la admisión, registro, atención médica, facturación y baja del paciente dentro del centro clínico.	
Actores: Actor Primario: Médico (verificado), PersonalAdministrativo (verificado) Actor Secundario: PersonalFacturacion (verificado)	
Precondiciones: 1. El usuario debe haber completado el caso de uso <Verificar usuario> con éxito. 2. El usuario debe tener un rol asignado dentro del sistema	
Flujo Normal: 1. El sistema ejecuta el caso de uso <Iniciar sesión> y <Verificar usuario>. 2. El sistema redirige al usuario al subsistema de Gestión Clínica al Paciente 3. El usuario accede a las funcionalidades correspondientes a su rol: - El Médico puede: acceder al historial clínico, añadir diagnósticos y tratamientos, notificar procedimientos de pago - El PersonalAdministrativo puede: registrar pacientes, gestionar admisiones, modificar datos, gestionar altas y bajas - El PersonalFacturacion puede: ver notificaciones de pago y gestionar la facturación 4. El usuario realiza las gestiones necesarias dentro del sistema 5. El usuario cierra su sesión al finalizar	
Flujo Alternativo: 3a. El usuario intenta acceder a una funcionalidad fuera de su rol 3a1. El sistema deniega el acceso y muestra un mensaje de error de permisos 3a2. El usuario puede continuar usando las funcionalidades permitidas para su rol	
Poscondiciones: Las gestiones realizadas por el usuario quedan registradas correctamente en el sistema según su rol	

Diagrama de Clases

Diagrama Antiguo

Este fue el primer diagrama de clases que realizamos, y a comparación del actual hay bastantes diferencias. El principal problema es que contenía clases que finalmente no se implementaron en el código, como **PilaAcciones** y **ColaPacientes**, las cuales en un principio pensamos que serían necesarias para gestionar el historial de acciones del personal administrativo y la cola de pacientes en espera, pero finalmente decidimos no incluirlas porque añadían una complejidad innecesaria para el alcance del proyecto.



Proyecto-Reto: Programación y Entornos de Desarrollo

Además de eso, el diagrama antiguo incluía atributos y métodos que no llegaron a existir en el código final, como **pacientesAtendidos** dentro de la clase **Medico**, **listarPacientesAtendidos()**, **buscarPacienteAtendido()**, o métodos dentro de **HistorialMedico** como **contieneAlergia()**, **buscarDiagnosticosPorPalabra()** y **getTotalAntecedentes()**, todos ellos presentes en el diagrama antiguo pero ausentes en la implementación real.

Otro problema era que algunos modificadores estaban mal colocados, y también había relaciones que ya no tenían validez, como la composición de **PersonalAdmin** con **PilaAcciones** y **ColaPacientes**, que al eliminarse esas clases dejaba de tener sentido.

La clase **EstadoPacienteException** en el diagrama antiguo aparecía con un atributo String mensaje que tampoco existía como tal en el código, ya que el mensaje se gestiona a través del constructor de RuntimeException.

Por todos estos motivos decidimos actualizar el diagrama para que fuera coherente con el código que realmente se ha implementado.

Diagrama Actual

El diagrama actual corrige todo lo mencionado anteriormente. Se han eliminado las clases **PilaAcciones** y **ColaPacientes** al no estar presentes en el código, y con ellas todas las relaciones y métodos que dependían de ellas.

Se han ajustado los atributos de cada clase para que coincidan exactamente con los que aparecen en el código, añadiendo también los métodos privados de validación que en el diagrama antiguo no estaban reflejados, como **validarDni()**, **validarTelefono()**, **validarCorreo()**, **validarEstado()** o **validarNumeroColegiado()**, entre otros. Estos métodos son importantes porque son los que garantizan la integridad de los datos dentro del sistema.

También se han corregido las relaciones entre clases, dejando únicamente aquellas que realmente existen en el código: la herencia de **Medico**, **PersonalAdmin** y **Paciente** respecto a **Persona**, la composición de **Paciente** con **HistorialMedico**, y las asociaciones de **PersonalAdmin** y **Medico** hacia **Paciente**. La dependencia de **AreaFacturacion** con **EstadoPacienteException** también se mantiene correctamente representada, ya que es la única excepción personalizada del sistema y se lanza directamente desde el método **generarFactura()**.

Clases del diagrama:

- **Clase Persona .**
- **Clase Medico.**
- **Clase PersonalAdmin.**

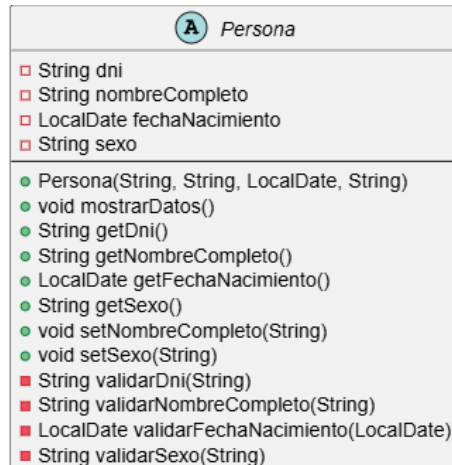
Proyecto-Reto: Programación y Entornos de Desarrollo

- Clase Paciente.
- Clase AreaFacturacion.
- Clase HistorialMedico.



Clase Persona

Esta clase es creada para que las clases Medico, PersonalAdmin y Paciente la hereden, ya que contendrá los datos generales que debe tener una persona.



Atributos:

- **String dni <<final>>**: contiene el número del DNI y es final ya que no se modificará posteriormente debido a que las personas solo pueden tener un único número del DNI. Es String ya que contiene tanto parte numérica como alfabética, por lo tanto siendo String puede contener ambas cosas de forma textual.
- **String NombreCompleto**: nombre y apellidos de la persona.
- **LocalDate fechaNacimiento <<fecha>>**: fecha de nacimiento de la persona. Es final porque no se puede cambiar.
- **String sexo**: el sexo de la persona es String en caso de que no sea simplemente masculino o femenino.

Métodos:

- **mostrarDatos**: método utilizado para mostrar todos los datos de los atributos de la clase Persona. Las clases hijas pueden sobrescribir este método (polimorfismo).
- **getDNI()**: getter del DNI.
- **getNombreCompleto()**: getter del nombre completo.
- **getFechaNacimiento()**: getter de la fecha de nacimiento.

Proyecto-Reto: Programación y Entornos de Desarrollo

- **getSexo():** getter del sexo.
- **setNombreCompleto(String):** setter del nombre completo (con validación).
- **setSexo(String):** setter del sexo (con validación).

Métodos privados de validación:

- **validarDni(String):** valida que el DNI tenga 9 caracteres (8 números + 1 letra).
- **validarNombreCompleto(String):** valida que el nombre no sea nulo.
- **validarFechaNacimiento(LocalDate):** valida que la fecha no sea nula ni futura.
- **validarSexo(String):** valida que el sexo sea 'M' o 'F'.

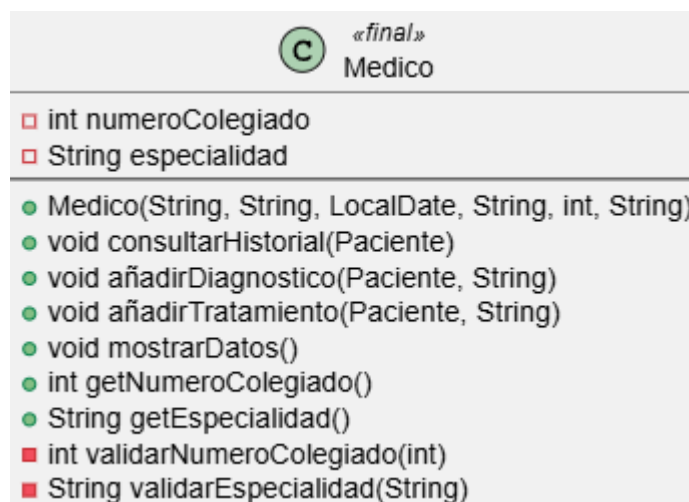
Relaciones entre clases:

La clase Persona se relaciona con las clases Medico, PersonalAdmin y Paciente ya que estas heredarán los atributos de la clase Persona.

Multiplicidades:

Debido a que aplicamos la herencia en las relaciones entre clases no hay multiplicidad.

Clase Medico



Proyecto-Reto: Programación y Entornos de Desarrollo

Esta clase hereda de Persona y representa a los médicos del hospital. Se encarga de atender pacientes, consultar historiales y añadir diagnósticos o tratamientos.

Atributos:

- **int numeroColegiado <<final>>**: número de colegiado del médico. Es final porque es único e inmutable. Es int porque es un número entero.
- **String especialidad**: especialidad médica del doctor (Cardiología, Pediatría, etc...). No es final porque puede cambiar si el médico se reespecializa.

Métodos:

- **consultarHistorial(Paciente)**: muestra el historial clínico completo del paciente.
- **añadirDiagnostico(Paciente, String)**: añade un diagnóstico al historial del paciente.
- **añadirTratamiento(Paciente, String)**: añade un tratamiento al historial del paciente.
- **mostrarDatos()**: sobrescribe el método de Persona para mostrar también numeroColegiado y especialidad.
- **getNumeroColegiado()**: getter del número de colegiado.
- **getEspecialidad()**: getter de la especialidad.

Métodos privados de validación:

- **validarNumeroColegiado(int)**: valida que el número de colegiado tenga 9 dígitos y solo contenga números.
- **validarEspecialidad(String)**: valida que la especialidad no sea nula ni esté vacía.

Relaciones entre clases:

Hereda de Persona.

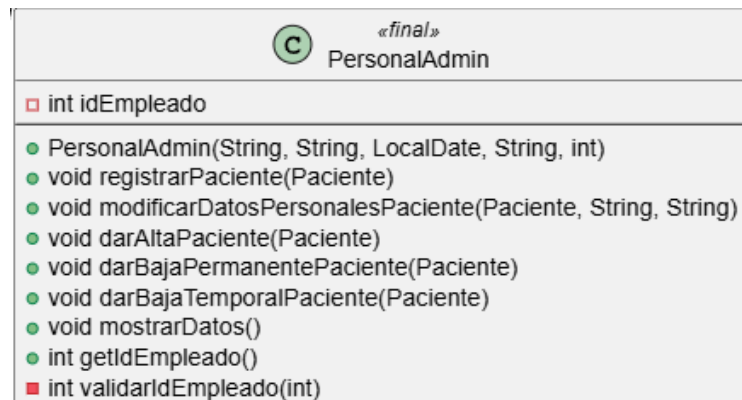
ATIENDE pacientes: Medico --> Paciente

Multiplicidades:

Un Medico ATIENDE muchos Paciente ($1 \rightarrow N$).

Un Paciente puede ser atendido por varios Medico ($N \rightarrow N$).

Clase PersonalAdmin



Esta clase hereda de **Persona** y representa al personal administrativo del hospital. Se encarga de gestionar el registro de pacientes, modificaciones de datos, altas y bajas.

Atributos:

- **int idEmpleado <<final>>**: identificador único del empleado. Es final porque no cambia después de asignarse. Es int porque es un número entero.

Métodos:

- **registrarPaciente(Paciente)**: registra un paciente en el sistema directamente.
- **modificarDatosPersonalesPaciente(Paciente, String, String)**: modifica los datos personales de un paciente (nombre completo y sexo).
- **darAltaPaciente(Paciente)**: da el alta médica a un paciente.
- **darBajaPermanentePaciente(Paciente)**: da de baja permanente a un paciente.
- **darBajaTemporalPaciente(Paciente)**: da de baja temporal a un paciente.
- **mostrarDatos()**: sobrescribe el método de **Persona** para mostrar también el **idEmpleado**.
- **getIdEmpleado()**: getter del id del empleado.

Métodos privados de validación:

- **validarIdEmpleado(int)**: valida que el código del empleado solo contenga números.

Proyecto-Reto: Programación y Entornos de Desarrollo

Relaciones entre clases:

Hereda de Persona.

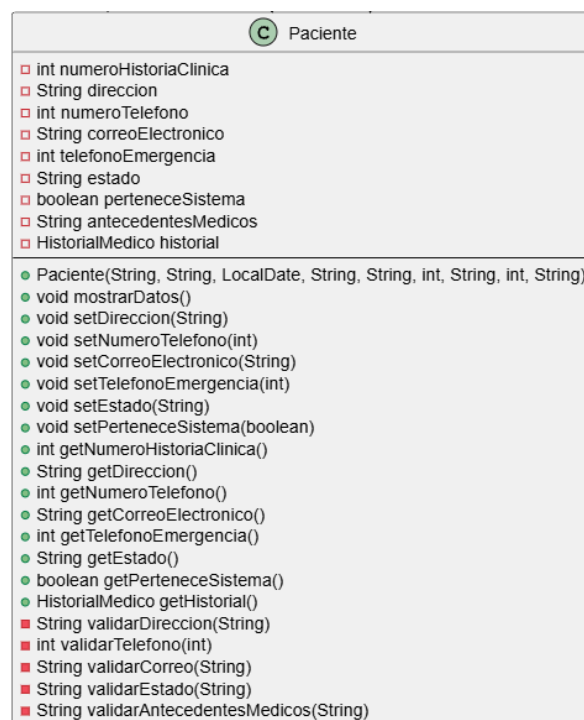
GESTIONA pacientes: PersonalAdmin --> Paciente

Multiplicidades:

Un PersonalAdmin GESTIONA muchos Paciente ($1 \rightarrow N$).

Un Paciente puede ser gestionado por un PersonalAdmin ($N \rightarrow 1$).

Clase Paciente



Esta clase hereda de Persona y representa a los pacientes del hospital. Contiene sus datos personales y su historial médico.

Atributos:

- **int numeroHistoriaClinica:** número único de historia clínica. Es int porque es un número entero.

Proyecto-Reto: Programación y Entornos de Desarrollo

- **String direccion:** dirección del paciente.
- **int numeroTelefono:** número de teléfono del paciente. Es int porque no necesita letras.
- **String correoElectronico:** correo electrónico del paciente.
- **int telefonoEmergencias:** teléfono de contacto de emergencia. Es int porque no necesita letras.
- **String estado:** estado del paciente. Puede ser: ACTIVO, ALTA_MEDICA, BAJA_TEMPORAL, BAJA_PERMANENTE.
- **boolean perteneceSistema:** indica si el paciente está activo en el sistema (true) o no (false).
- **HistorialMedico historial:** composición, el paciente tiene un historial médico. Si el paciente desaparece, su historial también.

Métodos:

- **mostrarDatos():** sobrescribe el método de Persona para mostrar también los datos específicos del paciente.
- Setters:
 - **setDireccion(String),**
 - **setNumeroTelefono(int),**
 - **setCorreoElectronico(String),**
 - **setTelefonoEmergencia(int),**
 - **setEstado(String),**
 - **setPerteneceSistema(boolean).**
- Getters:
 - **getNumeroHistoriaClinica(),**
 - **getDireccion(),**
 - **getNumeroTelefono(),**
 - **getCorreoElectronico(),**
 - **getTelefonoEmergencia(),**
 - **getEstado(),**
 - **getPerteneceSistema(),**
 - **getHistorial().**

Proyecto-Reto: Programación y Entornos de Desarrollo

Métodos privados de validación:

- **validarDireccion(String)**: valida que la dirección no sea nula.
- **validarTelefono(int)**: valida que el teléfono tenga 9 dígitos.
- **validarCorreo(String)**: valida que el correo contenga '@'.
- **validarEstado(String)**: valida que el estado sea uno de los permitidos.
- **validarAntecedentesMedicos(String)**: valida que los antecedentes no sean nulos.

Relaciones entre clases:

Hereda de Persona.

POSEE historial médico: Paciente *-- HistorialMedico (composición).

GESTIONADO POR personal administrativo: PersonalAdmin --> Paciente

ATENDIDO POR médicos: Medico --> Paciente

FACTURADO POR área de facturación: AreaFacturacion --> Paciente

Multiplicidades:

Un Paciente POSEE un HistorialMedico ($1 \rightarrow 1$).

Un Paciente es GESTIONADO por un PersonalAdmin ($N \rightarrow 1$).

Un Paciente es ATENDIDO por varios Medico ($N \rightarrow N$).

Un Paciente es FACTURADO por un AreaFacturacion ($N \rightarrow 1$).

Clase AreaFacturacion

Esta clase maneja todas las facturas del hospital. Es una clase independiente con atributos estáticos para que todas las facturas compartan la misma lista y el mismo contador.

C	AreaFacturacion
□	int idFactura
□	String concepto
□	double importe
□	LocalDate fechaFactura
□	static List<AreaFacturacion> facturas
□	static Map<Integer, List<AreaFacturacion>> facturasPorPaciente
□	static int contadorFacturas
●	AreaFacturacion(String, double, LocalDate)
●	void generarFactura(Paciente) throws EstadoPacienteException
●	static List<AreaFacturacion> buscarFacturasPorPaciente(int)
●	static void notificarProcedimiento(Paciente, String, double)
●	static void listarTodasFacturas()
●	static void mostrarFacturasDePaciente(Paciente)
●	String toString()
●	int getIdFactura()
●	String getConcepto()
●	double getImporte()
●	LocalDate getFechaFactura()
■	int validarIdFactura(int)
■	String validarConcepto(String)
■	double validarImporte(double)
■	LocalDate validarFecha(LocalDate)

Atributos:

- **int idFactura <<final>>**: identificador único de la factura. Es final porque no cambia después de crearse.
- **String concepto**: concepto de la factura (ej: "Consulta médica", "Análisis de sangre").
- **double importe**: cantidad a pagar. No puede ser negativa.
- **LocalDate fechaFactura**: fecha de emisión de la factura.
- **static List<AreaFacturacion> facturas**: lista estática donde se guardan TODAS las facturas del sistema. Es static para que todas las facturas se añadan a la misma lista.
- **static Map<Integer, List<AreaFacturacion>> facturasPorPaciente**: mapa estático que asocia cada paciente (por su número de historia clínica) con su lista de facturas. Permite búsqueda rápida.
- **static int contadorFacturas**: contador estático para generar IDs únicos de factura. Es static para que cada factura tenga un ID diferente (no reiniciar el contador por cada factura).

Proyecto-Reto: Programación y Entornos de Desarrollo

Métodos:

- **AreaFacturacion(String, double, LocalDate)**: constructor que recibe concepto, importe y fecha (el ID se genera automáticamente).
- **generarFactura(Paciente) throws EstadoPacienteException**: genera una factura y la asocia al paciente. Lanza excepción si el paciente está de baja temporal o permanente.
- **static List<AreaFacturacion> buscarFacturasPorPaciente(int)**: busca las facturas de un paciente por su número de historia clínica usando el mapa.
- **static void notificarProcedimiento(Paciente, String, double)**: notifica un procedimiento con importe específico.
- **static void listarTodasFacturas()**: muestra todas las facturas del sistema.
- **static void mostrarFacturasDePaciente(Paciente)**: muestra las facturas de un paciente específico.
- **toString()**: devuelve una representación textual de la factura.
- **Getters:**
 - **getIdFactura()**,
 - **getConcepto()**,
 - **getImporte()**,
 - **getFechaFactura()**.

Métodos privados de validación:

- **validarIdFactura(int)**: valida que el ID sea mayor que 0.
- **validarConcepto(String)**: valida que el concepto no sea nulo ni vacío.
- **validarImporte(double)**: valida que el importe sea mayor que 0.
- **validarFecha(LocalDate)**: valida que la fecha no sea nula ni futura.

Relaciones entre clases:

FACTURA a pacientes: AreaFacturacion --> Paciente

LANZA EstadoPacienteException: AreaFacturacion ..> EstadoPacienteException

NOTIFICA AreaFacturacion ..> Medico

Multiplicidades:

Un AreaFacturacion FACTURA a muchos Paciente (1 → N).

Un Paciente puede tener muchas AreaFacturacion (N → N).

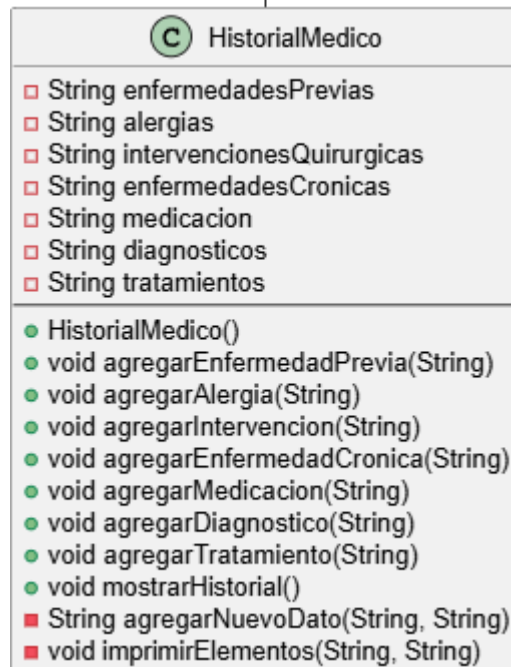
Medico notificará a AreaFacturacion en caso de que sea necesario (1 → 1)

Proyecto-Reto: Programación y Entornos de Desarrollo

La excepción es lanzada cuando un paciente está de baja temporal o permanente.

Clase HistorialMedico

Esta clase guarda todo el historial médico de un paciente. Es una clase que está en composición con Paciente, lo que significa que pertenece exclusivamente a un paciente..



Atributos:

Todos los atributos son de tipo `String` y utilizan un formato con separador `|` para almacenar múltiples valores. Por ejemplo: "Asma | Diabetes | ".

- **String enfermedadesPrevias:** enfermedades que ha tenido el paciente anteriormente.
- **String alergias:** alergias del paciente (medicamentos, alimentos, etc.).
- **String intervencionesQuirurgicas:** cirugías a las que se ha sometido el paciente.
- **String enfermedadesCronicas:** enfermedades de larga duración (hipertensión, diabetes, etc.).
- **String medicacion:** medicamentos que toma el paciente actualmente.
- **String diagnosticos:** diagnósticos realizados por los médicos.
- **String tratamientos:** tratamientos prescritos al paciente.

Métodos:

- **agregarEnfermedadPrevia(String), agregarAlergia(String), agregarIntervencion(String), agregarEnfermedadCronica(String), agregarMedicacion(String), agregarDiagnostico(String), agregarTratamiento(String)**: métodos para añadir elementos a cada campo usando el separador.
- **mostrarHistorial()**: muestra todo el historial clínico de forma ordenada.

Métodos privados de validación:

- **agregarNuevoDato(String, String)**: añade un elemento a un String con separadores.
- **imprimirElementos(String, String)**: imprime todos los elementos de un String con separadores.

Relaciones entre clases:

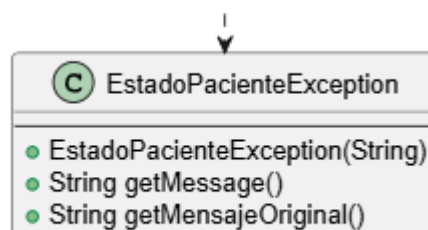
POSEE por Paciente (composición): Paciente *-- HistorialMedico

Multiplicidades:

Un HistorialMedico es POSEE por un Paciente (1 → 1).

Clase EstadoPacienteException

Esta es una excepción personalizada que se utiliza para manejar errores relacionados con el estado del paciente.



Características especiales:

Hereda de **RuntimeException (unchecked)**, lo que significa que no obliga a manejarla con try-catch, aunque el código sí lo hace.

Proyecto-Reto: Programación y Entornos de Desarrollo

Se lanza en `AreaFacturacion.generarFactura()` cuando el paciente está de baja temporal o permanente.

Se captura en `AreaFacturacion.notificarProcedimiento()` para que el programa no se detenga.

Métodos:

- **EstadoPacienteException(String)**: constructor que recibe el mensaje de error y lo pasa a la clase padre `RuntimeException`.
- **getMessage()**: método sobrescrito que devuelve el mensaje con un prefijo [ERROR ESTADO PACIENTE] para mayor claridad.
- **getMensajeOriginal()**: devuelve el mensaje de error sin el prefijo.

Relaciones entre clases:

LANZADA POR `AreaFacturacion`: `AreaFacturacion` ..> `EstadoPacienteException`

Multiplicidades:

No aplica multiplicidad porque es una excepción.

Diagramas de Comportamiento

Diagrama de Actividad:

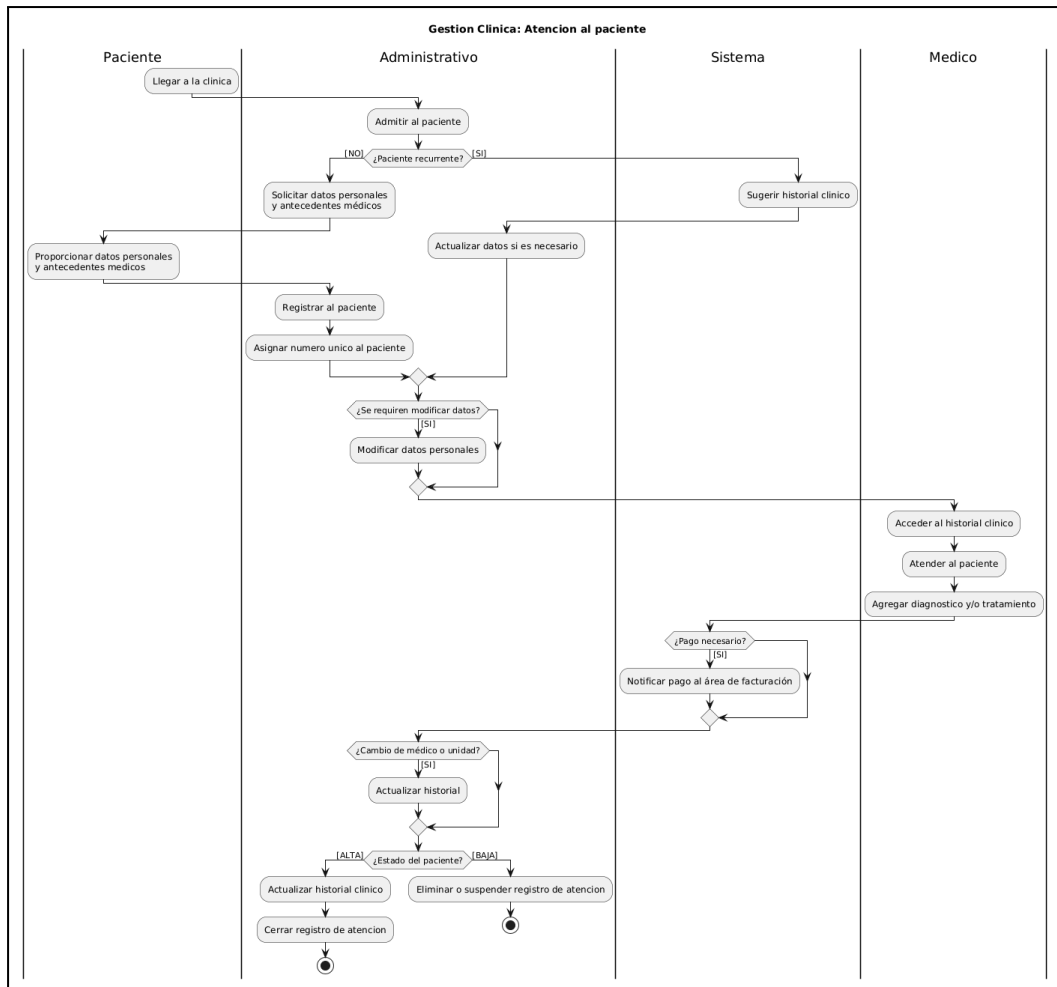
En el diagrama de actividad hemos creado 2 prototipos, el primero fue corregido con el profesor, estaba incompleto y no profundiza correctamente con la solución que hemos planteado.

Además de que hemos utilizado también al paciente dentro del diagrama cosa que hemos corregido en los demás, ya que el paciente no interactúa directamente con el sistema, sino que el sistema envuelve las necesidades y las gestiones del propio.

ANTIGUO DIAGRAMA DE ACTIVIDAD

Este es la imagen del antiguo diagrama de actividad, a comparación de el diagrama actual hay varias incongruencias, la primera es que a los pacientes que se están registrando se les cuestiona la modificación de datos y da la opción, cuando lo normal es introducirlo correctamente y cuando se realizan cambios es cuando ese paciente registrado ya tiene la opción de modificarlo si existe algún error.

Proyecto-Reto: Programación y Entornos de Desarrollo



Otro error fue añadir **Sistema** como un actor o alguien que ejecuta decisiones, cuando estamos trabajando dentro del propio sistema y quienes toman decisiones de lo que está sucediendo dentro de él son aquellas personas que realmente interactúan en él como médico, personal administrativo o personal de facturación.

Una de las decisiones modeladas que no tiene mucho sentido es **actualizar datos si es necesario**, no aporta mucho y al final es incensario teniendo el **if ¿Se requiere modificación de datos?**, es muy redundante.

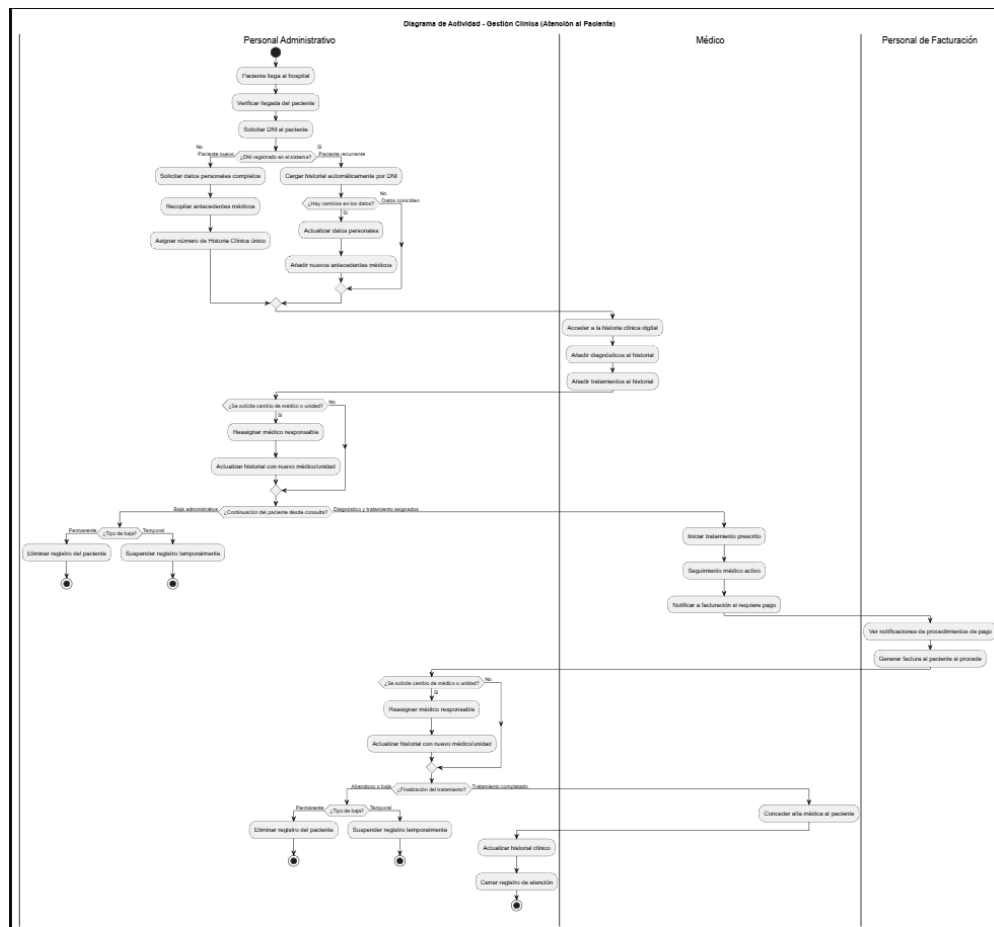
Sobre no profundizar, nos referimos al apartado de Médico, dentro del programa hace falta una de las decisiones que si la implementamos en el programa, menciona el enunciado y si que es necesaria y es **Notificar al área de facturación si requiere pago**, en caso de que el médico crea que es necesario dependiendo de la atención o servicio suministrados al paciente.

Otra modificación que es necesaria es que es el médico el único que puede dar el alta médica al paciente, y el personal administrativo en ese caso solo se encargaría de cerrar el registro de atención únicamente.

También hablando con Nacho nos dimos cuenta que el paciente puede decidir darse la baja administrativa o temporal tanto antes de la consulta, como después, incluso durante el

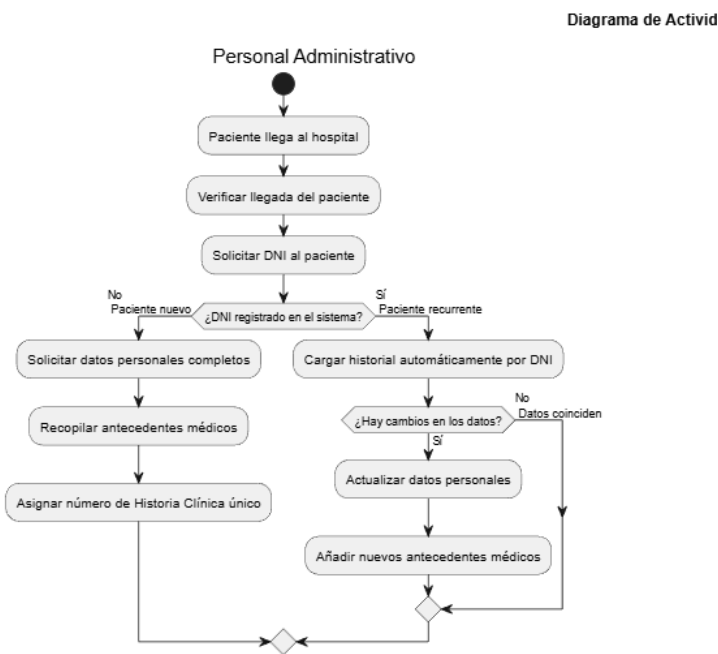
Proyecto-Reto: Programación y Entornos de Desarrollo

tratamiento, el paciente puede decidir darse la baja, ya sea por motivos personales o por insatisfacción con el tratamiento asignado, sin embargo esto no aparece reflejado en el antiguo diagrama.



El diagrama está dividido en 5 partes fundamentales para el funcionamiento correcto del programa.

El primer apartado es el siguiente:



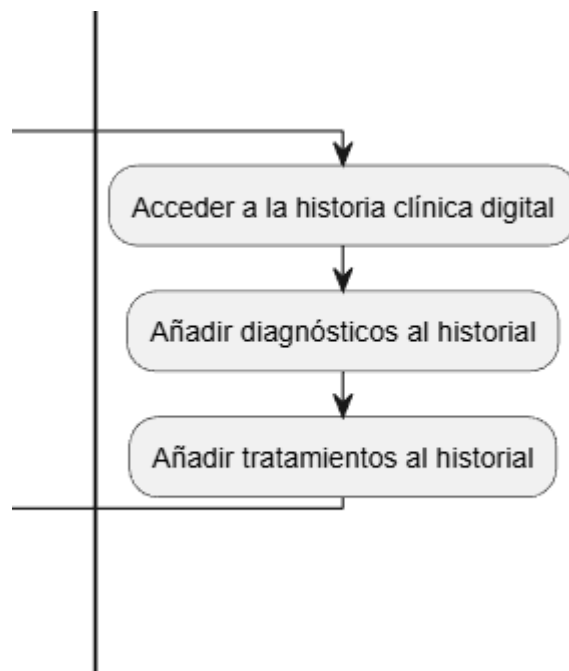
Proyecto-Reto: Programación y Entornos de Desarrollo

El protagonista en esta parte del diagrama es el personal Administrativo, ya que es quien en un principio recibe al paciente, primero verifica la llegada del paciente solicitando su DNI, lo introduce en el sistema.

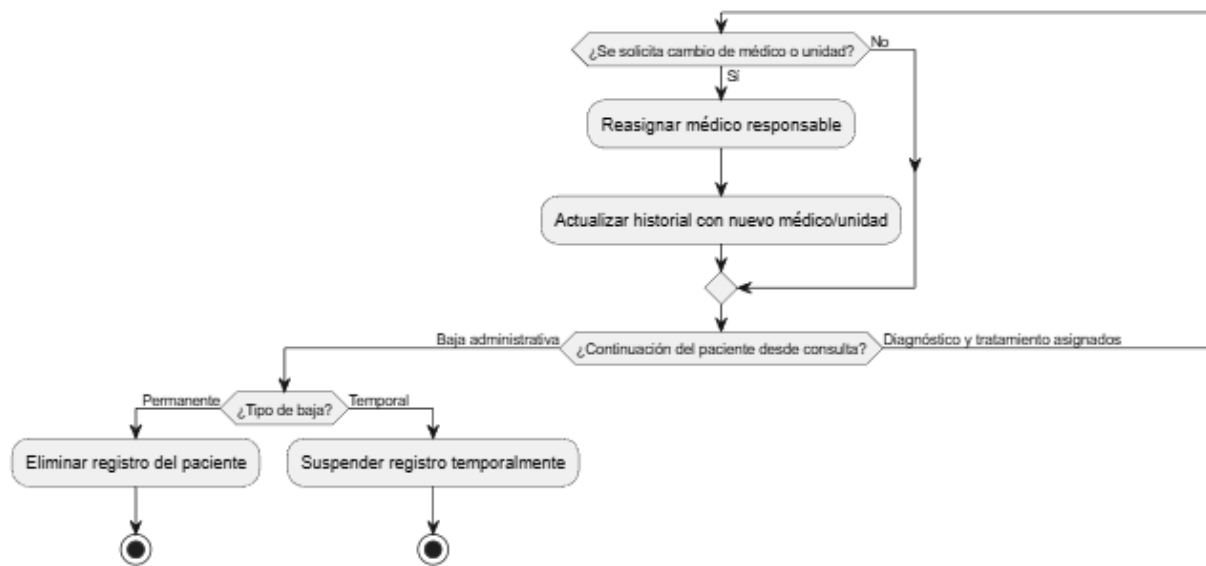
A partir de este punto se divide el flujo en 2, la primera, que el DNI si está registrado, en este caso, se cargará de forma automática el historial del paciente, si existe un error en sus datos existe la posibilidad de poder modificarlos, en caso de que no pasaría al siguiente punto.

En el caso de que el DNI no esté registrado en el sistema, se va a crear un nuevo usuario en donde el personal de administración rellenará con la información que se solicite y que le tendrá que dar el paciente para que se efectúe correctamente el registro.

Al completarlo, el sistema le asignará un identificador único para poder acceder de forma automatizada a su historial clínico para poder acceder correctamente a la consulta con el médico.



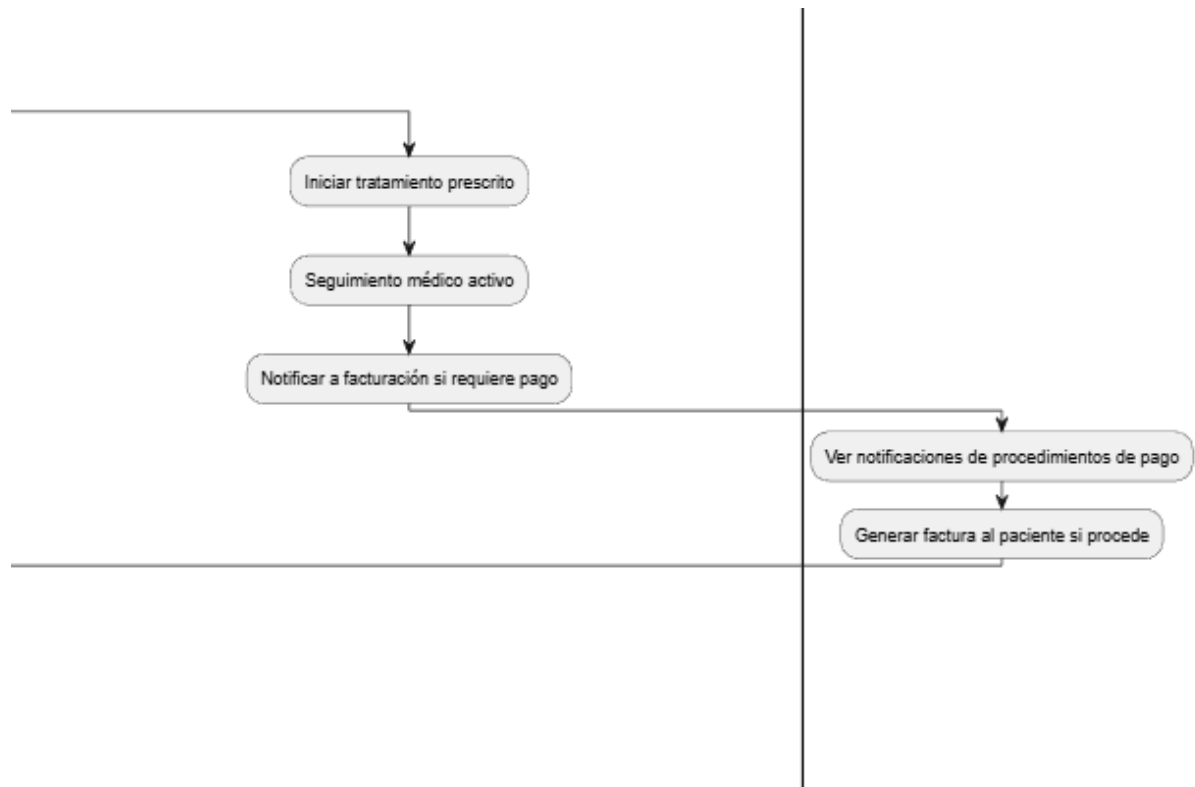
Ahora es el médico quien interactúa con el sistema, durante la consulta accede al historial con el DNI del paciente, dentro puede acceder diagnósticos y también tratamientos en caso de que la situación del paciente lo ameriten.



Volvemos con el Personal de Administración, durante la consulta con el médico, el paciente tiene la libertad total de escoger dependiendo de su satisfacción si quiere seguir tanto con el mismo médico, como cambiar de unidad o incluso darse de baja permanente del programa debido al tratamiento recibido por parte del médico.

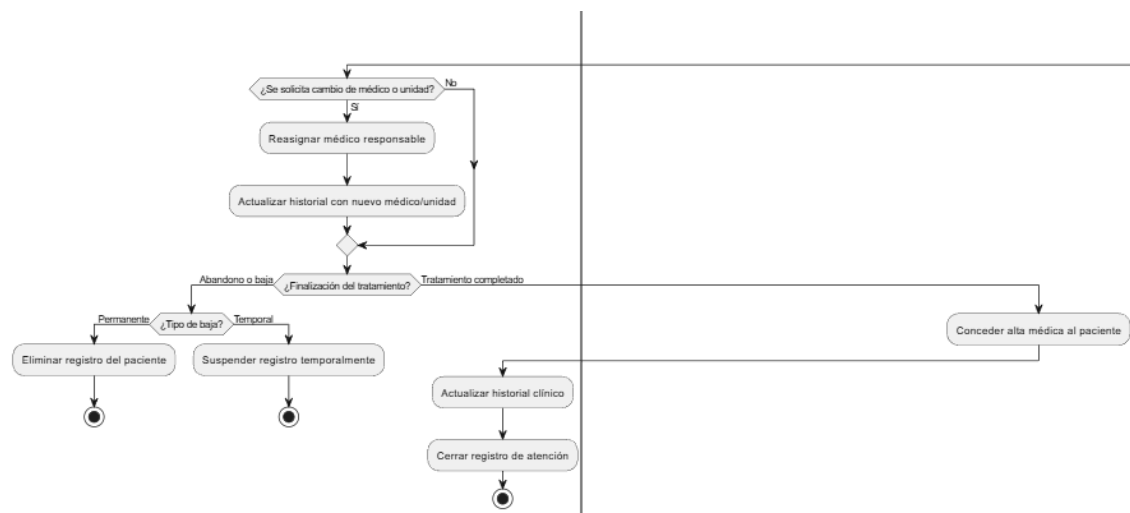
Eso en caso de que no quede satisfecho con la atención recibida por lo que el flujo acabaría en este punto, por otro lado en caso de estar contento con la atención de seguir con el tratamiento recibido por el médico nos lleva al siguiente punto.

Proyecto-Reto: Programación y Entornos de Desarrollo



En este punto del flujo de decisiones el médico puede iniciar el tratamiento al paciente, al que se le realizará un seguimiento activo, en caso de que el tratamiento requiera una cuota el médico debe de enviarle una notificación de cobro al área de facturación.

En este apartado el área de facturación deberá estar pendiente de la bandeja de notificaciones de pago y desde aquí generar una factura al paciente con el cobro respectivo



Terminando con el flujo el paciente puede decidir durante el proceso de tratamiento el cambio de unidad de médico en caso de que vea que el tratamiento no surge efecto y puede al igual que antes cambiarse de médico, darse de baja o seguir con el tratamiento

Proyecto-Reto: Programación y Entornos de Desarrollo

donde el médico cuando lo vea necesario concederá el alta médica del paciente, se actualiza los datos y se cierra el registro al cliente .

Diagrama de Estados:

Se han realizado distintos diagramas de Estados por lo tanto hemos decidido mostrar tanto el diagrama Previo como el actual explicando porque se han realizado esas modificaciones en el actual diagrama de Estados.

Diagrama de Estados Previo

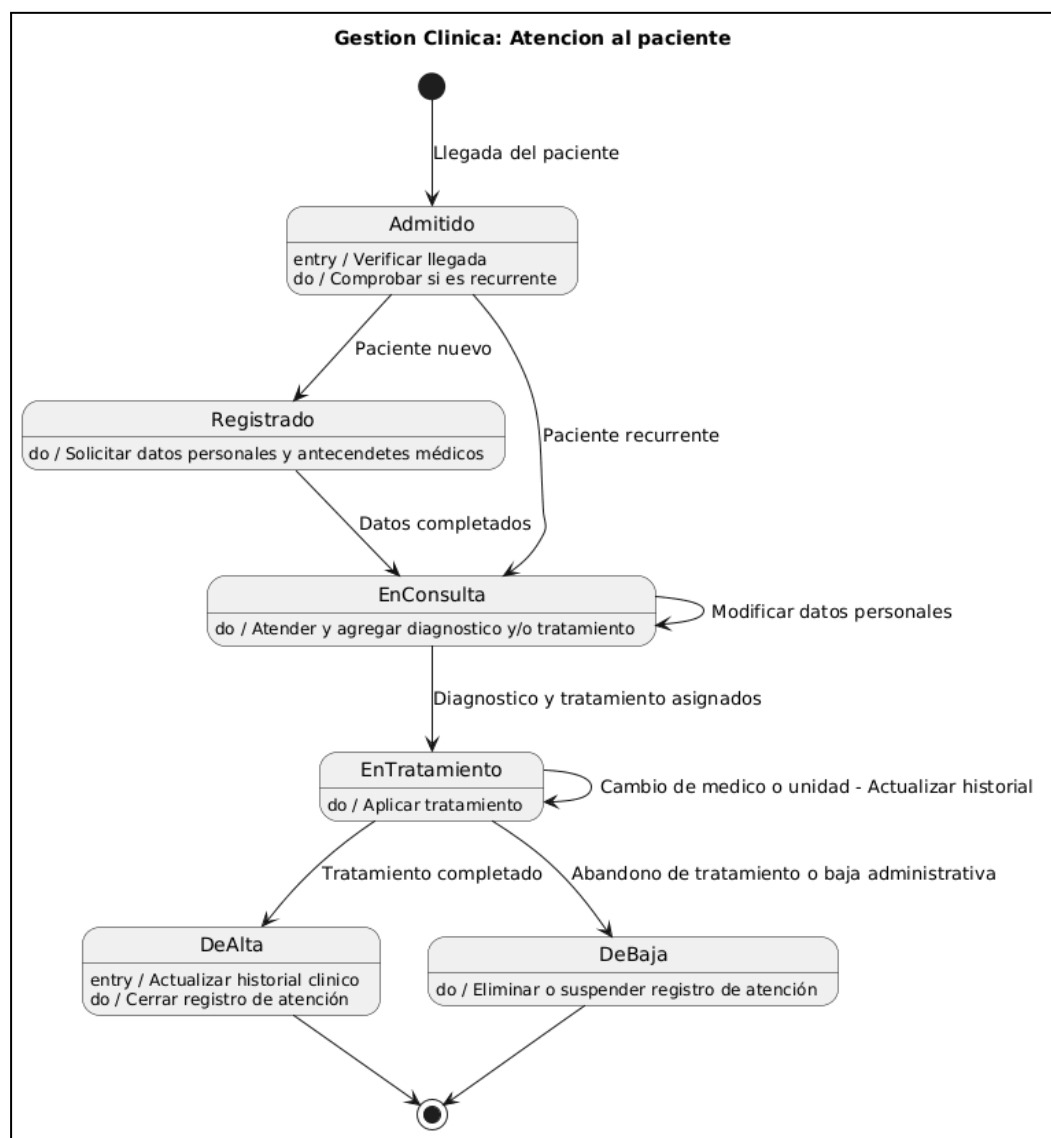


Diagrama de Estados Actual

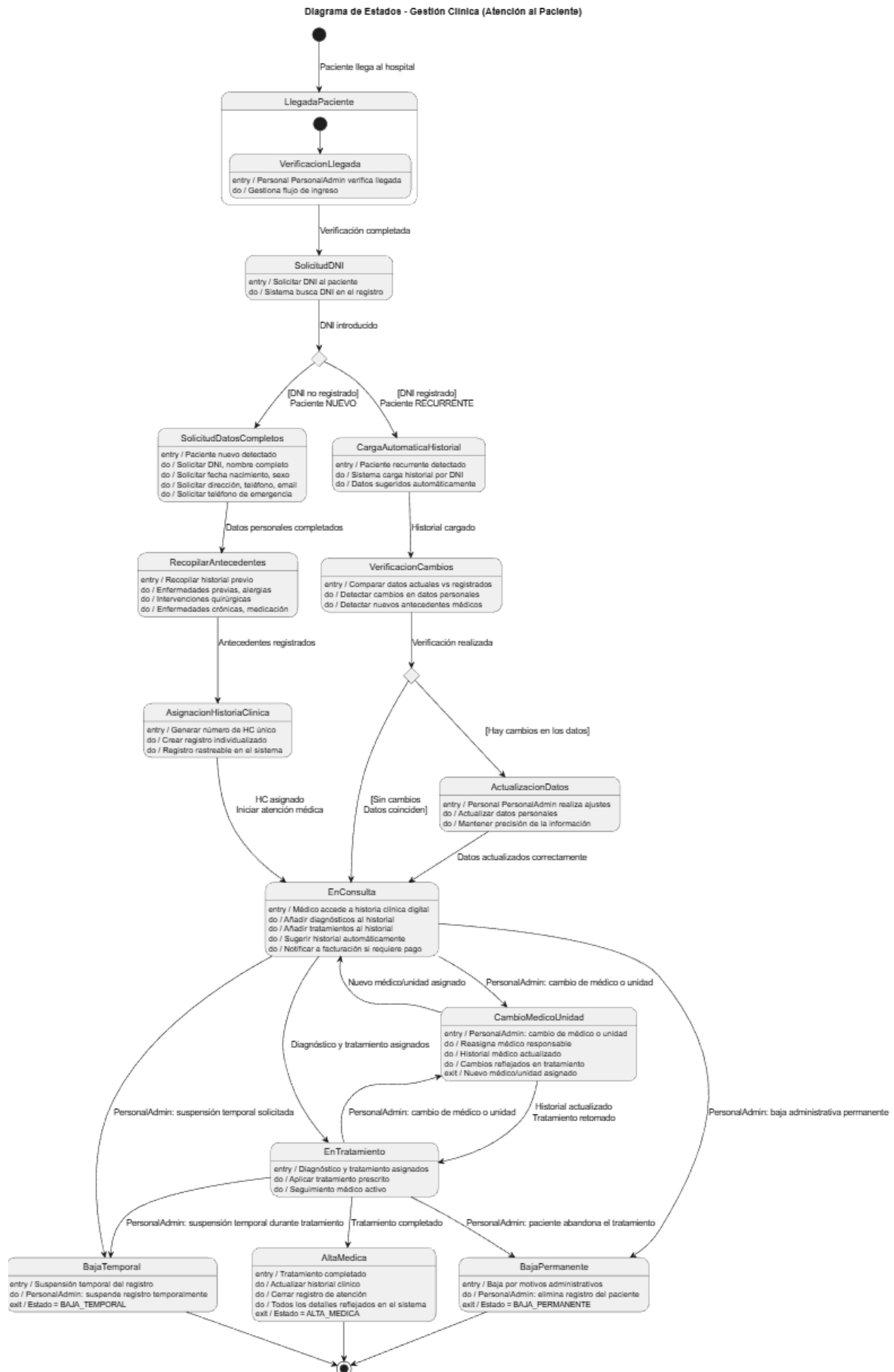
Este es nuestro diagrama de estado actual, a comparación del anterior hemos realizado varias modificaciones, el diagrama anterior era demasiado simplificado a comparación del actual.

No trataba directamente varios de los puntos los cuales son muy importantes para el flujo correcto de la atención al paciente dentro del programa, también estaban incorrectos algunos nombres de los estados, y se centraba principalmente en que los pacientes tenían que recibir un tratamiento más que en la atención al paciente en general.

Debido a todos estos motivos hemos optado por la realización del siguiente diagrama de Estados el cual corrige todo lo dicho anteriormente.

Representa el ciclo del paciente dentro del sistema, este diagrama muestra los estados por los que pasa un paciente dentro del sistema de gestión clínica, así como los eventos que provocan cada transición entre estados.

Proyecto-Reto: Programación y Entornos de Desarrollo



Proyecto-Reto: Programación y Entornos de Desarrollo

Estados:

LlegadaPaciente: el flujo comienza con la llegada del paciente al hospital, donde el personal administrativo verifica su llegada y gestiona el flujo de ingreso.

SolicitudDNI: se solicita el DNI al paciente y el sistema lo busca en el registro para determinar si ya existe o no.

SolicitudDatosCompleto: estado exclusivo para pacientes nuevos. Se solicitan todos los datos personales necesarios: DNI, nombre completo, fecha de nacimiento, sexo, dirección, teléfono, correo electrónico y teléfono de emergencia.

RecopilarAntecedentes: una vez recogidos los datos personales del paciente nuevo, se recopilan sus antecedentes médicos: enfermedades previas, alergias, intervenciones quirúrgicas, enfermedades crónicas y medicación.

AsignacionHistoriaClinica: estado exclusivo para pacientes nuevos. Se genera un número de historia clínica único y se crea un registro individualizado y rastreable dentro del sistema.

CargaAutomaticaHistorial: estado exclusivo para pacientes recurrentes. El sistema carga automáticamente el historial asociado al DNI introducido, sugiriendo los datos ya existentes.

VerificacionCambios: se comparan los datos actuales del paciente recurrente con los que ya estaban registrados, detectando posibles cambios en datos personales o nuevos antecedentes médicos.

ActualizacionDatos: si se detectan cambios, el personal administrativo actualiza los datos personales y añade los nuevos antecedentes para mantener la precisión de la información.

EnConsulta: el médico accede a la historia clínica digital del paciente, añade diagnósticos y tratamientos, y el sistema notifica al área de facturación si el procedimiento requiere pago.

CambioMedicoUnidad: el personal administrativo gestiona el cambio de médico o de unidad, reasignando al médico responsable y actualizando el historial para reflejar los cambios en el tratamiento. Puede ocurrir tanto durante la consulta como durante el tratamiento.

EnTratamiento: el paciente sigue el tratamiento prescrito por el médico, quien lleva el control y el seguimiento activo del mismo.

AltaMedica: el tratamiento ha sido completado. Se actualiza el historial clínico, se cierra el registro de atención y todos los detalles quedan reflejados en el sistema.

BajaPermanente: el paciente es dado de baja de forma definitiva por motivos administrativos o por decisión propia. El registro queda eliminado del sistema y `PerteneceSistema` pasa a false.

BajaTemporal: el registro del paciente queda suspendido temporalmente según las políticas del hospital. `PerteneceSistema` se mantiene en true.

Transiciones dentro del diagrama:

Inicio → **LlegadaPaciente**: el paciente llega al hospital.

LlegadaPaciente → **SolicitudDNI**: verificación completada.

SolicitudDNI → **SolicitudDatosCompletos**: DNI no registrado, paciente nuevo.

SolicitudDNI → **CargaAutomaticaHistorial**: DNI registrado, paciente recurrente.

SolicitudDatosCompletos → **RecopilarAntecedentes**: datos personales completados.

RecopilarAntecedentes → **AsignacionHistoriaClinica**: antecedentes registrados.

AsignacionHistoriaClinica → **EnConsulta**: HC asignado, iniciar atención médica.

CargaAutomaticaHistorial → **VerificacionCambios**: historial cargado.

VerificacionCambios → **ActualizacionDatos**: hay cambios en los datos.

VerificacionCambios → **EnConsulta**: sin cambios, datos coinciden.

ActualizacionDatos → **EnConsulta**: datos actualizados correctamente.

EnConsulta → **CambioMedicoUnidad**: cambio de médico o unidad.

CambioMedicoUnidad → **EnConsulta**: nuevo médico/unidad asignado.

EnConsulta → **EnTratamiento**: diagnóstico y tratamiento asignados.

EnTratamiento → **CambioMedicoUnidad**: cambio de médico o unidad.

CambioMedicoUnidad → **EnTratamiento**: historial actualizado, tratamiento retomado.

EnTratamiento → **AltaMedica**: tratamiento completado.

EnConsulta → **BajaPermanente**: baja administrativa permanente.

EnConsulta → **BajaTemporal**: suspensión temporal solicitada.

EnTratamiento → **BajaPermanente**: el paciente abandona el tratamiento.

EnTratamiento → **BajaTemporal**: suspensión temporal durante el tratamiento.

AltaMedica → **Fin**: registro cerrado.

BajaPermanente → **Fin**: registro eliminado.

BajaTemporal → **Fin**: registro suspendido.

PLANTUML

Aquí se colocarán los plantUml de los diagramas, tanto los nuevos como también los viejos, con la finalidad de tener todo organizado.

Para diferenciarlos mejor el código viejo llevara el fondo de color negro, para distinguir mejor los diagramas nuevos y viejos

Diagrama de caso de usos

No hay, este fue realizado en draw.io

Diagrama de actividad

Diagrama antiguo

```
@startuml
title Gestion Clinica: Atencion al paciente

|Paciente|
:Llegar a la clinica;

|Administrativo|
:Admitir al paciente;

if (¿Paciente recurrente?) then ([NO])
    |Administrativo|
    :Solicitar datos personales
    y antecedentes médicos;

    |Paciente|
    :Proporcionar datos personales
    y antecedentes medicos;

    |Administrativo|
    :Registrar al paciente;
    :Asignar numero unico al paciente;
else ([SI])
    |Sistema|
    :Sugerir historial clinico;

    |Administrativo|
    :Actualizar datos si es necesario;
end if;
```

```
|Administrativo|
if (¿Se requieren modificar datos?) then ([SI])
    :Modificar datos personales;
end if;

|Medico|
:Acceder al historial clinico;
:Atender al paciente;
:Agregar diagnostico y/o tratamiento;

|Sistema|
if (¿Pago necesario?) then ([SI])
    :Notificar pago al área de facturación;
end if;

|Administrativo|
if (¿Cambio de médico o unidad?) then ([SI])
    :Actualizar historial;
end if;

if (¿Estado del paciente?) then ([ALTA])
    :Actualizar historial clinico;
    :Cerrar registro de atencion;
    stop
else ([BAJA])
    :Eliminar o suspender registro de atencion;
    stop
end if;
@enduml
```

Diagrama nuevo

```
@startuml
title Diagrama de Actividad - Gestión Clínica (Atención al Paciente)
```

```
|Personal Administrativo|
start

:Paciente llega al hospital;
:Verificar llegada del paciente;
:Solicitar DNI al paciente;

if (¿DNI registrado en el sistema?) then (No \n Paciente nuevo)

    :Solicitar datos personales completos;

    :Recopilar antecedentes médicos;

    :Asignar número de Historia Clínica único;

else (Sí \n Paciente recurrente)

    :Cargar historial automáticamente por DNI;

    if (¿Hay cambios en los datos?) then (Sí)
        :Actualizar datos personales;
        :Añadir nuevos antecedentes médicos;

    else (No \n Datos coinciden)

    endif

endif

' El médico lleva el control clínico y de modificaciones dentro del historial clínico del paciente
' El admin gestiona los cambios administrativos del paciente

|Médico|

:Acceder a la historia clínica digital;
:Añadir diagnósticos al historial;
:Añadir tratamientos al historial;

|Personal Administrativo|

if (¿Se solicita cambio de médico o unidad?) then (Sí)

    :Reasignar médico responsable;
    :Actualizar historial con nuevo médico/unidad;
```



```
else (No)

endif

if (¿Continuación del paciente desde consulta?) then (Baja administrativa)

    if (¿Tipo de baja?) then (Permanente)

        :Eliminar registro del paciente;

        stop

    else (Temporal)

        :Suspender registro temporalmente;

        stop

    endif

else (Diagnóstico y tratamiento asignados)

    ' El médico aplica y hace seguimiento al paciente para ver si el tratamiento
    realiza el efecto
    ' esperado por el medico

    |Médico|

    :Iniciar tratamiento prescrito;
    :Seguimiento médico activo;

    ' El sistema notifica automáticamente a facturación
    ' si el procedimiento requiere pago.
    ' Facturación lo consulta desde su propio menú

    :Notificar a facturación si requiere pago;

    |Personal de Facturación|

    ' El personal de Facturación es quien revisa si se ha enviado una
    notificación de cobro al paciente
    ' esa notificación es enviada de parte del médico en caso de se crea
    conveniente dependiendo de si
    ' después de la consulta y tratamiento resulta necesario

    :Ver notificaciones de procedimientos de pago;
```

```
:Generar factura al paciente si procede;

|Personal Administrativo|

if (¿Se solicita cambio de médico o unidad?) then (Sí)
    :Reasignar médico responsable;
    :Actualizar historial con nuevo médico/unidad;
else (No)

endif

if (¿Finalización del tratamiento?) then (Abandono o baja)

    if (¿Tipo de baja?) then (Permanente)

        :Eliminar registro del paciente;

        stop
    else (Temporal)

        :Suspender registro temporalmente;

        stop
    endif

else (Tratamiento completado)

    |Médico|
    :Conceder alta médica al paciente;

    ' El medico es el único después de una evaluación al paciente si se
    Le puede realizar el alta medica

    |Personal Administrativo|
    :Actualizar historial clínico;
    :Cerrar registro de atención;

    stop

endif

endif

@enduml
```

Diagrama de estados

Diagrama antiguo

```
@startuml
title Gestion Clinica: Atencion al paciente

[*] --> Admitido : Llegada del paciente

state Admitido{
    Admitido : entry / Verificar llegada
    Admitido : do / Comprobar si es recurrente
}

state Registrado{
    Registrado : do / Solicitar datos personales y antecedentes médicos
}

state EnConsulta {
    EnConsulta : do / Atender y agregar diagnostico y/o tratamiento
}

state EnTratamiento {
    EnTratamiento : do / Aplicar tratamiento
}

state DeAlta {
    DeAlta : entry / Actualizar historial clinico
    DeAlta : do / Cerrar registro de atención
}

state DeBaja {
    DeBaja : do / Eliminar o suspender registro de atención
}

Admitido --> Registrado : Paciente nuevo
Admitido --> EnConsulta : Paciente recurrente

Registrado --> EnConsulta : Datos completados

EnConsulta --> EnConsulta : Modificar datos personales
EnConsulta --> EnTratamiento : Diagnostico y tratamiento asignados
```

```
EnTratamiento --> EnTratamiento : Cambio de medico o unidad - Actualizar
historial
EnTratamiento --> DeAlta : Tratamiento completado
EnTratamiento --> DeBaja : Abandono de tratamiento o baja administrativa

DeAlta --> [*]
DeBaja --> [*]
@enduml
```

Diagrama nuevo

```
@startuml

title Diagrama de Estados - Gestión Clínica (Atención al Paciente)

[*] --> LlegadaPaciente : Paciente llega al hospital

state LlegadaPaciente {
    [*] --> VerificacionLlegada
    VerificacionLlegada : entry / Personal PersonalAdmin verifica
llegada
    VerificacionLlegada : do / Gestiona flujo de ingreso
}

state SolicitudDNI {
    SolicitudDNI : entry / Solicitar DNI al paciente
    SolicitudDNI : do / Sistema busca DNI en el registro
}

LlegadaPaciente --> SolicitudDNI : Verificación completada

state "¿DNI registrado?" as DecisionDNI <<choice>>
SolicitudDNI --> DecisionDNI : DNI introducido

' =====
'          RAMA PACIENTE NUEVO
' =====

' Aquí solo estarán aquellos pacientes los cuales no se encuentren
registados
' en el sistema, sera necesario para registrarlos con efectividad todos
los datos
' que sean totalmente necesarios
```

```
state SolicitudDatosCompletos {

  SolicitudDatosCompletos : entry / Paciente nuevo detectado
  SolicitudDatosCompletos : do / Solicitar DNI, nombre completo
  SolicitudDatosCompletos : do / Solicitar fecha nacimiento, sexo
  SolicitudDatosCompletos : do / Solicitar dirección, teléfono, email
  SolicitudDatosCompletos : do / Solicitar teléfono de emergencia

}

state RecopilarAntecedentes {

  RecopilarAntecedentes : entry / Recopilar historial previo
  RecopilarAntecedentes : do / Enfermedades previas, alergias
  RecopilarAntecedentes : do / Intervenciones quirúrgicas
  RecopilarAntecedentes : do / Enfermedades crónicas, medicación

}

state AsignacionHistoriaClinica {

  AsignacionHistoriaClinica : entry / Generar número de HC único
  AsignacionHistoriaClinica : do / Crear registro individualizado
  AsignacionHistoriaClinica : do / Registro rastreable en el sistema

}

DecisionDNI --> SolicitudDatosCompletos : [DNI no registrado]\nPaciente
NUEVO
SolicitudDatosCompletos --> RecopilarAntecedentes : Datos personales
completados
RecopilarAntecedentes --> AsignacionHistoriaClinica : Antecedentes
registrados

' =====
'          RAMA PACIENTE RECURRENTE
' =====

' En esta rama se encontrara el flujo de aquellos pacientes los cuales
se
' encuentren registrados correctamente dentro del sistema

state CargaAutomaticaHistorial {

  CargaAutomaticaHistorial : entry / Paciente recurrente detectado
```

```
CargaAutomaticaHistorial : do / Sistema carga historial por DNI
CargaAutomaticaHistorial : do / Datos sugeridos automáticamente

}

state VerificacionCambios {

  VerificacionCambios : entry / Comparar datos actuales vs registrados
  VerificacionCambios : do / Detectar cambios en datos personales
  VerificacionCambios : do / Detectar nuevos antecedentes médicos

}

state "¿Hay cambios?" as DecisionCambios <<choice>>

state ActualizacionDatos {

  ActualizacionDatos : entry / Personal PersonalAdmin realiza ajustes
  ActualizacionDatos : do / Actualizar datos personales
  ActualizacionDatos : do / Mantener precisión de la información

}

DecisionDNI --> CargaAutomaticaHistorial : [DNI registrado]\nPaciente
RECORRENTE
CargaAutomaticaHistorial --> VerificacionCambios : Historial cargado
VerificacionCambios --> DecisionCambios : Verificación realizada
DecisionCambios --> ActualizacionDatos : [Hay cambios en los datos]
DecisionCambios --> EnConsulta : [Sin cambios\nDatos coinciden]
ActualizacionDatos --> EnConsulta : Datos actualizados correctamente

' El cambio de médico y de unidad en el problema no especifica cuando se
puede cambiar
' por lo que nosotros interpretamos por razones obvias dentro de
nuestras experiencias
' obviamente se puede solicitar el cambio de unidad estando en consulta
o mientras estas
' en tratamiento, al menos en la sanidad publica, de esta gestión se
encargaría el personal administrativo.

state CambioMedicoUnidad {

  CambioMedicoUnidad : entry / PersonalAdmin: cambio de médico o
unidad
  CambioMedicoUnidad : do / Reasigna médico responsable
```

```
CambioMedicoUnidad : do / Historial médico actualizado
CambioMedicoUnidad : do / Cambios reflejados en tratamiento
CambioMedicoUnidad : exit / Nuevo médico/unidad asignado

}

' =====
'           EN CONSULTA
' =====

' Una vez que ambos tipos de pacientes se encuentren registrados
correctamente en el sistema
' podrán asisistir correctamente a la consulta medica

state EnConsulta {

    EnConsulta : entry / Médico accede a historia clínica digital
    EnConsulta : do / Añadir diagnósticos al historial
    EnConsulta : do / Añadir tratamientos al historial
    EnConsulta : do / Sugerir historial automáticamente
    EnConsulta : do / Notificar a facturación si requiere pago
}

AsignacionHistoriaClinica --> EnConsulta : HC asignado\nIniciar atención
médica

EnConsulta --> CambioMedicoUnidad : PersonalAdmin: cambio de médico o
unidad
CambioMedicoUnidad --> EnConsulta : Nuevo médico/unidad asignado

' =====
'           EN TRATAMIENTO
' =====

' El tratamiento es impuesto unicamente por el médico, es quien llevará
el control
' y el tiempo que debe de seguirlo cada uno de los pacientes que se
encuentran bajo su cuidado

state EnTratamiento {

    EnTratamiento : entry / Diagnóstico y tratamiento asignados
    EnTratamiento : do / Aplicar tratamiento prescrito
    EnTratamiento : do / Seguimiento médico activo
```

```
}

EnConsulta --> EnTratamiento : Diagnóstico y tratamiento asignados

' El admin también puede hacer cambios durante el tratamiento
EnTratamiento --> CambioMedicoUnidad : PersonalAdmin: cambio de médico o
unidad
CambioMedicoUnidad --> EnTratamiento : Historial
actualizado\nTratamiento retomado

' =====
' ESTADOS FINALES
' =====

' Estos últimos estados son gestionados por el personal administrativo,
en donde por decisión propia
' del paciente decide si realizar la baja permanente o una baja
temporal, ahora bien el alta medica
' solo se la puede dar el medico si ha visto que se ha solucionado el
problema por el cual el paciente decidió
' realizar una consulta y el tratamiento fue completado.

state AltaMedica {

    AltaMedica : entry / Tratamiento completado
    AltaMedica : do / Actualizar historial clínico
    AltaMedica : do / Cerrar registro de atención
    AltaMedica : do / Todos los detalles reflejados en el sistema
    AltaMedica : exit / Estado = ALTA_MEDICA
}

state BajaPermanente {

    BajaPermanente : entry / Baja por motivos administrativos
    BajaPermanente : do / PersonalAdmin: elimina registro del paciente
    BajaPermanente : exit / Estado = BAJA_PERMANENTE
}

state BajaTemporal {

    BajaTemporal : entry / Suspensión temporal del registro
    BajaTemporal : do / PersonalAdmin: suspende registro temporalmente
    BajaTemporal : exit / Estado = BAJA_TEMPORAL
```



```
}

EnTratamiento --> AltaMedica : Tratamiento completado
EnConsulta --> BajaPermanente : PersonalAdmin: baja administrativa
permanente
EnTratamiento --> BajaPermanente : PersonalAdmin: paciente abandona el
tratamiento
EnConsulta --> BajaTemporal : PersonalAdmin: suspensión temporal
solicitada
EnTratamiento --> BajaTemporal : PersonalAdmin: suspensión temporal
durante tratamiento

AltaMedica --> [*]
BajaPermanente --> [*]
BajaTemporal --> [*]

@enduml
```

Diagrama de clases

Diagrama antiguo

```
@startuml

' Clase Abstracta Persona
abstract class Persona {
    - String dni
    - String nombreCompleto
    - LocalDate fechaNacimiento
    - String sexo
    --
    + void mostrarDatos()
    + String getDni()
    + String getNombreCompleto()
    + LocalDate getFechaNacimiento()
    + String getSexo()
}

' Clase PersonalAdmin
class PersonalAdmin <<final>> {
    - int idEmpleado
    - ColaPacientes colaPacientesRegistro
    - PilaAcciones historialAcciones
    --
}
```

```
+ void registrarPaciente(Paciente)
+ Paciente atenderSiguientePaciente()
+ void modificarDatosPaciente(Paciente, String, int, String, int)
+ void darAltaPaciente(Paciente)
+ void darBajaPaciente(Paciente, boolean)
+ void mostrarDatos()
+ int getIdEmpleado()
+ static void mostrarHistorialAcciones()
+ static int getTamanoCola()
}
```

' Clase Interna Pila

```
class PilaAcciones {
    - int MAXIMO
    - String[] v
    - int cima
    --
    + PilaAcciones(int)
    + PilaAcciones()
    + void apilar(String)
    + String desapilar()
    + String verTope()
    + boolean estaVacia()
    + void mostrarTodas()
    + int getTamano()
}
```

' Clase Interna Cola

```
class ColaPacientes {
    - int MAXIMO
    - Paciente[] v
    - int ini
    - int fin
    --
    + ColaPacientes(int)
    + ColaPacientes()
    + void encolar(Paciente)
    + Paciente desencolar()
    + boolean estaVacia()
    + boolean estaLlena()
    + int getTamano()
    + Paciente verPrimero()
}
```

' Clase Medico

```
class Medico <<final>> {
    - int numeroColegiado
    - String especialidad
    - "List<Paciente>" pacientesAtendidos
    --
    + void consultarHistorial(Paciente)
    + void añadirDiagnostico(Paciente, String)
    + void añadirTratamiento(Paciente, String)
    + Paciente buscarPacienteAtendido(int)
    + void listarPacientesAtendidos()
    + void mostrarDatos()
    + int getNumeroColegiado()
    + String getEspecialidad()
    + void setEspecialidad(String)
}
```

' Clase Paciente

```
class Paciente {
    - int numeroHistoriaClinica
    - String direccion
    - int numeroTelefono
    - String correoElectronico
    - int telefonoEmergencia
    - String estado
    - boolean activo
    - HistorialMedico historial
    --
    + void mostrarDatos()
    + void setDireccion(String)
    + void setNumeroTelefono(int)
    + void setCorreoElectronico(String)
    + void setTelefonoEmergencia(int)
    + void setEstado(String)
    + void setActivo(boolean)
    + int getNumeroHistoriaClinica()
    + String getDireccion()
    + int getNumeroTelefono()
    + String getCorreoElectronico()
    + int getTelefonoEmergencia()
    + String getEstado()
    + boolean isActivo()
    + HistorialMedico getHistorial()
}
```

' Clase Historial

```
class HistorialMedico {
```

```
- String enfermedadesPrevias
- String alergias
- String intervencionesQuirurgicas
- String enfermedadesCronicas
- String medicacion
- String diagnosticos
- String tratamientos
--
+ void agregarEnfermedadPrevia(String)
+ void agregarAlergia(String)
+ void agregarIntervencion(String)
+ void agregarEnfermedadCronica(String)
+ void agregarMedicacion(String)
+ void agregarDiagnostico(String)
+ void agregarTratamiento(String)
+ boolean contieneAlergia(String)
+ "List<String>" buscarDiagnosticosPorPalabra(String)
+ int getTotalAntecedentes()
+ void mostrarHistorial()
}

' Clase AreaFacturacion

class AreaFacturacion {
    - int idFactura
    - String concepto
    - double importe
    - LocalDate fechaFactura
    --
    - "List<AreaFacturacion>" facturas
    - "Map<Integer, List<AreaFacturacion>>" facturasPorPaciente
    - int contadorFacturas
    --
    + void generarFactura(Paciente) ' throws EstadoPacienteException
    + static double getFacturacionTotal()
    + static "List<AreaFacturacion>" buscarFacturasPorPaciente(int)
    + static void notificarProcedimiento(Paciente, String)
    + static void notificarProcedimiento(Paciente, String, double)
    + static void listarTodasFacturas()
    + static void mostrarFacturasDePaciente(Paciente)
    + String toString()
    + int getIdFactura()
    + String getConcepto()
    + double getImporte()
    + LocalDate getFechaFactura()
}

' Excepcion Personalizada (Clase Separada)
```

```
class EstadoPacienteException {
    - String mensaje
    --
    + EstadoPacienteException(String)
    + String getMessage()
}

' Herencias

Persona <|-- PersonalAdmin
Persona <|-- Medico
Persona <|-- Paciente

' Composiciones

Paciente *-- HistorialMedico : POSEE
PersonalAdmin *-- PilaAcciones
PersonalAdmin *-- ColaPacientes

' Asociaciones

PersonalAdmin --> Paciente : GESTIONA
Medico --> Paciente : ATIENDE
AreaFacturacion --> Paciente : FACTURA

' Dependencias

Medico ..> HistorialMedico : MODIFICA
Medico ..> AreaFacturacion : NOTIFICA
AreaFacturacion ..> EstadoPacienteException : lanza
AreaFacturacion ..> Paciente : usa

@enduml
```

Diagrama nuevo

@startuml

```
' Clase Abstracta Persona

abstract class Persona {
    - String dni
    - String nombreCompleto
    - LocalDate fechaNacimiento
    - String sexo
```

```
--
+ Persona(String, String, LocalDate, String)
+ void mostrarDatos()
+ String getDni()
+ String getNombreCompleto()
+ LocalDate getFechaNacimiento()
+ String getSexo()
+ void setNombreCompleto(String)
+ void setSexo(String)
- String validarDni(String)
- String validarNombreCompleto(String)
- LocalDate validarFechaNacimiento(LocalDate)
- String validarSexo(String)
}

' Clase PersonalAdmin

class PersonalAdmin <<final>> {
    - int idEmpleado
    --
    + PersonalAdmin(String, String, LocalDate, String, int)
    + void registrarPaciente(Paciente)
    + void modificarDatosPersonalesPaciente(Paciente, String, String)
    + void darAltaPaciente(Paciente)
    + void darBajaPermanentePaciente(Paciente)
    + void darBajaTemporalPaciente(Paciente)
    + void mostrarDatos()
    + int getIdEmpleado()
    - int validarIdEmpleado(int)
}

' Clase Medico

class Medico <<final>> {
    - int numeroColegiado
    - String especialidad
    --
    + Medico(String, String, LocalDate, String, int, String)
    + void consultarHistorial(Paciente)
    + void añadirDiagnostico(Paciente, String)
    + void añadirTratamiento(Paciente, String)
    + void mostrarDatos()
```

```
+ int getNumeroColegiado()
+ String getEspecialidad()
- int validarNumeroColegiado(int)
- String validarEspecialidad(String)
}

' Clase Paciente

class Paciente {
    - int numeroHistoriaClinica
    - String direccion
    - int numeroTelefono
    - String correoElectronico
    - int telefonoEmergencia
    - String estado
    - boolean perteneceSistema
    - String antecedentesMedicos
    - HistorialMedico historial
    --
    + Paciente(String, String, LocalDate, String, String, int, String, int,
String)
    + void mostrarDatos()
    + void setDireccion(String)
    + void setNumeroTelefono(int)
    + void setCorreoElectronico(String)
    + void setTelefonoEmergencia(int)
    + void setEstado(String)
    + void setPerteneceSistema(boolean)
    + int getNumeroHistoriaClinica()
    + String getDireccion()
    + int getNumeroTelefono()
    + String getCorreoElectronico()
    + int getTelefonoEmergencia()
    + String getEstado()
    + boolean getPerteneceSistema()
    + HistorialMedico getHistorial()
    - String validarDireccion(String)
    - int validarTelefono(int)
    - String validarCorreo(String)
    - String validarEstado(String)
    - String validarAntecedentesMedicos(String)
}
```

```
' Clase HistorialMedico

class HistorialMedico {
    - String enfermedadesPrevias
    - String alergias
    - String intervencionesQuirurgicas
    - String enfermedadesCronicas
    - String medicacion
    - String diagnosticos
    - String tratamientos
    --
    + HistorialMedico()
    + void agregarEnfermedadPrevia(String)
    + void agregarAlergia(String)
    + void agregarIntervencion(String)
    + void agregarEnfermedadCronica(String)
    + void agregarMedicacion(String)
    + void agregarDiagnostico(String)
    + void agregarTratamiento(String)
    + void mostrarHistorial()
    - String agregarNuevoDato(String, String)
    - void imprimirElementos(String, String)
}

' Clase AreaFacturacion

class AreaFacturacion {
    - int idFactura
    - String concepto
    - double importe
    - LocalDate fechaFactura
    - static List<AreaFacturacion> facturas
    - static Map<Integer, List<AreaFacturacion>> facturasPorPaciente
    - static int contadorFacturas
    --
    + AreaFacturacion(String, double, LocalDate)
    + void generarFactura(Paciente) throws EstadoPacienteException
    + static List<AreaFacturacion> buscarFacturasPorPaciente(int)
    + static void notificarProcedimiento(Paciente, String, double)
    + static void listarTodasFacturas()
    + static void mostrarFacturasDePaciente(Paciente)
```



```
+ String toString()
+ int getIdFactura()
+ String getConcepto()
+ double getImporte()
+ LocalDate getFechaFactura()
- int validarIdFactura(int)
- String validarConcepto(String)
- double validarImporte(double)
- LocalDate validarFecha(LocalDate)
}

' Clase EstadoPacienteException (RuntimeException)

class EstadoPacienteException {
    --
    + EstadoPacienteException(String)
    + String getMessage()
    + String getMessageOriginal()
}

' Herencias
Persona <|-- PersonalAdmin
Persona <|-- Medico
Persona <|-- Paciente

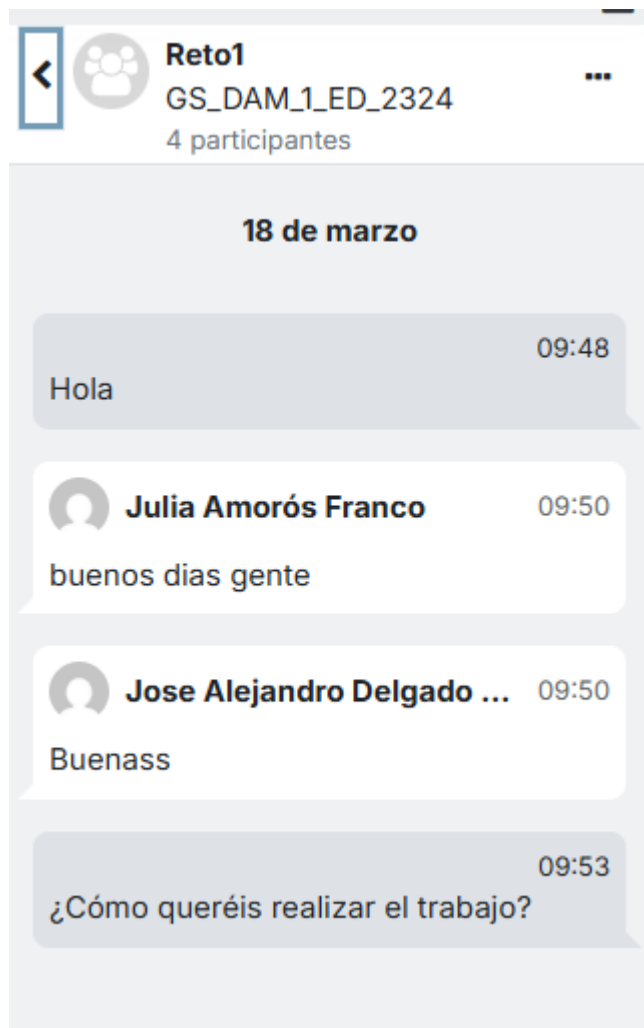
' Composición (relación fuerte TODO-PARTE)
Paciente *-- HistorialMedico : POSEE

' Asociaciones (relaciones de uso)
PersonalAdmin --> Paciente : GESTIONA
Medico --> Paciente : ATIENDE
AreaFacturacion --> Paciente : FACTURA

' Dependencias (uso temporal)
AreaFacturacion ..> EstadoPacienteException : LANZA
Medico ..> AreaFacturacion : NOTIFICA

@enduml
```

Conversaciones



Proyecto-Reto: Programación y Entornos de Desarrollo

Julia, ATSUSHINICA

22 de marzo de 2026



LauraMishifu lost 22/03/2026 13:23

Buenas tardes, paso por aquí ya el diagrama de clases para hablarlo luego en la reunión



LauraMishifu lost 22/03/2026 13:31

O si lo preferís por no tener tiempo, podemos hablarlo mañana en clase, que toca programación y entornos



Julia 22/03/2026 17:31

pues a ver

igual por tiempo mejor mñn

porque a q hora seria sino hoy?



LauraMishifu lost 22/03/2026 17:31

Ns, dijimos que más tarde de las 7

Mejor mañana yo creo que tenemos tiempo en programación y entornos



Julia 22/03/2026 18:23

valee

23 de marzo de 2026



ATSUSHINICA 23/03/2026 19:37

Hola, os mando el link del repositorio de git para que mandéis la solicitud de colaboración y aceptarla y dejar eso listo esta parte.

<https://github.com/ATSUSHINICA/Proyecto>

Julia, ATSUSHINICA

Proyecto-Reto: Programación y Entornos de Desarrollo

Julia, ATSUSHINICA

Así que no te preocupes mucho, si no te has mira mi carpeta clase, que estamos poniendo tambien comentarios de porque se realizan las cosas y tal

ATSUSHINICA 29/04/2026 21:11
claro que me fio de tu trabajo, de hecho me he metido a revisar y lo he visto ahi, ademas el apartado de interes tambien va de mi parte
yo tengo también la parte de culpa, por no haber puesto mi empeño en el trabajo y haber dejado de pasar tanto tiempo, la parte de documentación que tengas pásamela y lo añado.
ya pido disculpas si se mal interpreto lo que he dicho, pero vamos solo quiero saber si vais a aportar en el trabajo, nada más, y viendo el tiempo que nos queda os quería preguntar si queréis hacerlo, hay mucha gente que se han rendido y han decidido ya no hacerlo directamente
y quería saber si era el mismo caso aqui, yo hoy trasnochare e igual me pierdo el día de mañana haciendo esto porque no queda mucho tiempo

ATSUSHINICA 29/04/2026 21:19
me gustaria que si alguna puede hacer una reunion rapida porque nose sobre que diagrama estamos trabajando
este es el diagrama de casos de uso que yo tengo, pero es con la lógica del diagrama de clases que hice

nose si avanzar con las fichas porque tendria que ver la logica tambien del diagrama de clases con el que al final vamos a trabajar

@ATSUSHINICA me gustaria que si alguna puede hacer una reunion rapida porque nose sobre que diagrama estamos trabajando

LauraMishifu 29/04/2026 21:25
Si que puedo y así te enseño más o menos con el compartir pantalla para que se me entienda a que me refiero

ATSUSHINICA 29/04/2026 21:26
porfavor

LauraMishifu ha iniciado una llamada que ha durado una hora. 29/04/2026 21:26

Julia, ATSUSHINICA

Tienes una llamada perdida de **ATSUSHINICA** que ha durado unos segundos. 29/04/2026 22:38

LauraMishifu 29/04/2026 22:38
?

ATSUSHINICA 29/04/2026 22:38
mala mia
error

LauraMishifu 29/04/2026 22:38
Otra reunión?
JASJSJA

ATSUSHINICA 29/04/2026 22:38
JAJAJAJAJ

Julia 29/04/2026 22:39
AJAJAJA

30 de abril de 2026

Julia 30/04/2026 20:18
Holaa, he dejado en el documento de google docs los dos diagramas, los pasaria por aqui pero no estoy en mi pc entonces no tengo discord. Si podeis decidme que opinais y si veis algo que esta mal o que se podria mejorar o renombrar me lo decís y lo cambio

4 de mayo de 2026

ATSUSHINICA 04/05/2026 16:59
Hola hoy he hablado con el de entornos, el apartado de diagrama de usos únicamente solo le queda una cosa más por corregir que me dijo hoy, acabare todas las fichas y me pondré a ver vuestra parte, nose si ya habéis acabado o aun os queda algo para así revisarlo y ponernos concentrados en dejar todo niquelado y hacer la presentación

ATSUSHINICA 04/05/2026 21:38
voy a hacer un pequeño cambio respecto al flujo al momento de acceder al historial medico del paciente pero pregunto, por si acaso ya se ha hecho, para aquellos pacientes los cuales no sean recurrentes pedir el numero de historial medico y el dni por ejemplo y los que si lo son solo el numero para que de esa forma digamos que al ser recurrentes sea mas facil acceder, pregunto porque estoy con las fichas y aun no me he leído el código pero igual ya lo hizo laura jjsjsjs
y tambien pasarme vuestros nombres completos porfa para ponerlos en las fichas xd

Julia 04/05/2026 21:40
vas a hacer un cambio en la clase?

Proyecto-Reto: Programación y Entornos de Desarrollo

Julia, ATSUSHINICA

LauraMishifu  04/05/2026 21:40
Ay va pues no he hecho esa separación de pacientes recurrentes y no recurrentes

@ATSUSHINICA y tambien pasarme vuestros nombres completos porfa para ponerlos en las fichas xd

LauraMishifu  04/05/2026 21:41
Laura Leciñena Andres

@LauraMishifu  Ay va pues no he hecho esa separación de pacientes recurrentes y no recurrentes

Julia 04/05/2026 21:41
bueno no pasa nada, se hace ahora y ya

@LauraMishifu  Ay va pues no he hecho esa separación de pacientes recurrentes y no recurrentes

ATSUSHINICA 04/05/2026 21:41
es que se me ha venido la iluminacion mientras estaba haciendo las fichas

@ATSUSHINICA y tambien pasarme vuestros nombres completos porfa para ponerlos en las fichas xd

Julia 04/05/2026 21:41
Julia Amorós Franco

LauraMishifu  Ay va pues no he hecho esa separación de pacientes recurrentes y no recurrentes

LauraMishifu  04/05/2026 21:43
Pero es que al ser un hospital tampoco le veo yo la necesidad de discernir entre recurrentes o no porque no es un hotel donde se vaya a dar una atención especial, ns si me explico

ATSUSHINICA 04/05/2026 21:43
sí, te entiendo, y pienso igual que tu

Julia 04/05/2026 21:43
ya pero como el enunciado lo menciona pues bueno

ATSUSHINICA 04/05/2026 21:44
me parece estúpido con perdon tener que hacer una division entre 2 tipos de pacientes, no es necesario crear una clase distinta dentro del main para poder acceder al historial en vez de validar 1 dato hacemos 2 para los que no son recurrentes y así los diferenciamos
a mí me parece bastante raro pero bueno, como lo pone en el enunciado segun cepero hay que hacerlo lo mas fiel a el, y la forma de no matarnos reconstruyendo todo es esa yo acabare hoy todas las fichas, si no te importa lo hago yo y así de esa forma veo todas las clases y hago el cambio que te digo y luego mando por aqui las modificaciones
Laura pasame tu nombre completo porfas

Julia, ATSUSHINICA

Laura pasame tu nombre completo porfas

LauraMishifu  04/05/2026 21:50
Okey, hazlo como tu prefieras, tranquilo, hoy he actualizado el diagrama de clases y creo que ya estaría bien, echale un ojo si puedes, me quedaria añadir esos cambios en la documentación que he puesto también las subclases

@ATSUSHINICA Laura pasame tu nombre completo porfas

LauraMishifu  04/05/2026 21:50
Laura Leciñena Andres

@LauraMishifu  Okey, hazlo como tu prefieras, tranquilo, hoy he actualizado el diagrama de clases y creo que ya estaría bien, echale un ojo si puedes, me quedaria añadir esos cambio...

ATSUSHINICA 04/05/2026 21:51
okey perfecto, cualquier cambio que haga lo mando por aqui para que lo sepas, descargare todo y lo subire a el main lo que tengamos los 3 para que todos luego cojamos lo mismo y no tengamos una cosa de una y otra de otra

LauraMishifu  04/05/2026 21:52
Vale, perfecto :3

@ATSUSHINICA okey perfecto, cualquier cambio que haga lo mando por aqui para que lo sepas, descargare todo y lo subire a el main lo que tengamos los 3 para que todos luego cojamos lo mis...

Julia 04/05/2026 21:56
vale muchas gracias 🙏

ATSUSHINICA 04/05/2026 22:06
nadaaaa, a vosotras

5 de mayo de 2026

ATSUSHINICA 05/05/2026 2:01
Acabo de terminar todas las fichas, organizaré todo el main, os recomiendo que las leáis, vienen bien para entender todo el flujo del programa, tuve que hacer unos cambios pero ya está modificado todo en la documentación

ATSUSHINICA 05/05/2026 2:41
En el main ya tenéis todo lo que hemos hecho junto, están los diagramas, capturas y demás
Las clases también están dentro, para que bajéis los cambios y tengan todo

Julia 05/05/2026 9:01
Holaa, una duda, es que estaba mirando los entregables del proyecto y en el apartado de persistencia de datos menciona almacenar informacion en ficheros de texto y carga automatica desde uno de ellos de algunos de los datos, y no sabia si eso está implementado en el codigo o no

Proyecto-Reto: Programación y Entornos de Desarrollo

Julia, ATSUSHINICA



ATSUSHINICA

05/05/2026 9:16

Me parece que no lo está

Es que hemos visto todo eso muy reciente, ahora tendremos que implementar y añadir cosas o cambiar

Yo tuve q cambiar bastantes cosas en el de casos jsjsjs



Julia

05/05/2026 9:24

vale vale era por saberlo jsjsj



ATSUSHINICA

05/05/2026 18:10

Hola buenas, hoy a las 20 podreis hacer una reunión rápida ?



Julia

05/05/2026 18:19

Por mi bien



ATSUSHINICA

05/05/2026 20:09

estais disponible para hacer la reunion?



Julia

05/05/2026 20:10

yo si



LauraMishifu

 lost

05/05/2026 20:10

yo también



ATSUSHINICA

ha iniciado una llamada que ha durado 31 minutos. 05/05/2026 20:10

6 de mayo de 2026



ATSUSHINICA

06/05/2026 11:05

Solo me queda modificar la clase historial médico

Y volver q hacer el main con el subsistema de verificación y ya lo tendríamos

Hacemos pruebas todos, jugamos con el

Luego actualizamos el diagrama de clases y lo documentamos



Julia

06/05/2026 15:43

Hola chicos, quería comentaros que como quiero repetir con programación y se necesita un mínimo de 11 horas suspendidas, he decidido suspender también entornos, para ello no iré a la presentación, ya que esa es la nota de la evaluación y no hay examen. Lo siento de verdad, cualquier cosa que necesitéis yo estoy encantada de ayudaros, si queréis por ejemplo probar programa o lo que sea, me lo decís y yo lo hago sin ningún problema.

Mañana se lo comentaré a Nacho, y de nuevo, de verdad que lo siento y espero que os salga bien la presentación.

Julia, ATSUSHINICA



Julia

06/05/2026 15:43

Hola chicos, quería comentaros que como quiero repetir con programación y se necesita un mínimo de 11 horas suspendidas, he decidido suspender también entornos, para ello no iré a la presentación, ya que esa es la nota de la evaluación y no hay examen. Lo siento de verdad, cualquier cosa que necesitéis yo estoy encantada de ayudaros, si queréis por ejemplo probar programa o lo que sea, me lo decís y yo lo hago sin ningún problema.

Mañana se lo comentaré a Nacho, y de nuevo, de verdad que lo siento y espero que os salga bien la presentación.



LauraMishifu

 lost

06/05/2026 16:08

Julia cualquier duda que tengas o cualquier problema estamos aquí para resolverlo, que se puede sacar, estoy segura de que si te pones lo sacas tia, estás segura de que no quieres hacer por lo menos el intento?



Julia

06/05/2026 16:13

No te preocupes, creo que la mejor opcion es volver a intentarlo el año q viene solo con programacion, por asi decirlo

Muchas gracias de todas formas



LauraMishifu

 lost

06/05/2026 16:30

Bueno, pues entonces al siguiente curso te lo sacas tia, que tú puedes



ATSUSHINICA

06/05/2026 17:35

Ostras no pasa nada, respeto tu decisión y agradezco primero que nos lo comentas con tiempo antes de la presentación y también el tiempo que le has dedicado al trabajo, opino como Laura si te pones seguro lo sacas, pero cada quien toma su decisión y es respetable, te deseo mucho ánimo y en verano mete la caña que así el año que viene se hace menos pesado



Julia

06/05/2026 17:45

sisi el año que viene a tope, muchas gracias 🙏



ATSUSHINICA

06/05/2026 20:00

Ya tenéis subido en el mi rama el programa completo por si queréis probarlo, ya con ficheros implementados

Terminaré de modificar cosas pero será en el main @LauraMishifu ya puedes hacer el diagrama de clases con las modificaciones que hice a cada clase, en general estaban bien, simplemente las adapte más a lo que pedían

Probarlo y si encontráis bugs os lo agradecería



LauraMishifu

 lost

06/05/2026 20:11

Okey, en cuanto llegue a casa lo hago



ATSUSHINICA

06/05/2026 20:20

Todas las clases las modifiqué a mano pero el main lo hice con IA, aún así lo optimizaré

Me parecen mucho 1.4k líneas de código

Pero eso da igual porque no entra en el diagrama de clases

71

 **ATSUSHINICA**

5 de mayo de 2026

 **ATSUSHINICA** 05/05/2026 18:31
Laura estoy con el código, si puedes hacer llamada te lo agradecería jeje

 **LauraMishifu**  05/05/2026 18:32
Hostia pues no tengo nada de voz, ns yo si se me escuchará
JAJSJJASA

 **ATSUSHINICA** 05/05/2026 18:32
no hay problema solo es para comentarte unas cosas que he visto
que estoy con el código

 **LauraMishifu** ha iniciado una llamada que ha durado 5 minutos. 05/05/2026 18:33

7 de mayo de 2026

 **ATSUSHINICA** ayer a las 10:18
Ya está subido corregido
Ya puedes hacer tranquila el diagrama
Ya no modificó más, que se haga lo que dios quiera

 **LauraMishifu**  ayer a las 12:27
JAJSJAJSJSJA
Okey
Yo luego voy a corregir el diagrama de Julia por la tarde, cuando lo tenga te aviso
Y termino el diagrama de clases

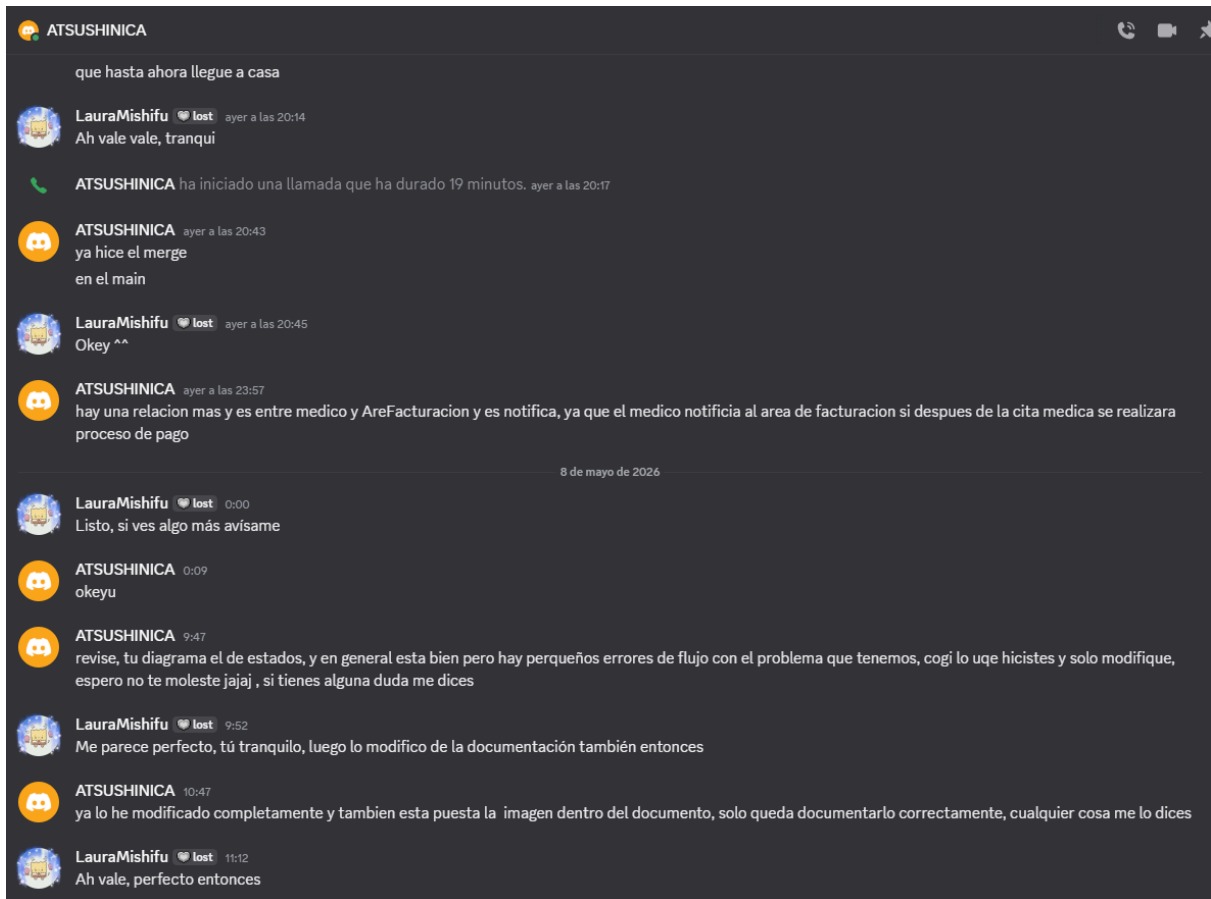
 **ATSUSHINICA** ayer a las 20:05
Laura puedes hacer llamada en 5 minutos ?

 **LauraMishifu** ha iniciado una llamada que ha durado unos segundos. ayer a las 20:06

 **LauraMishifu**  ayer a las 20:06
Me he adelantado perdon
estás?

 **ATSUSHINICA** ayer a las 20:14
voy esoty calentando la comida

Proyecto-Reto: Programación y Entornos de Desarrollo



Proyecto-Reto: Programación y Entornos de Desarrollo

 **ATSUSHINICA**

 **ATSUSHINICA** 11:51
si quieres te paso la foto del diagrama de estados para que en cada transiciones de añadas su respectivo trozo y si quieres explicarlo un poco mas mejor porque me quede corto sjjsjs
na se ve igual de mal, mejor pillas el plant y le vas poniendo capturas
queda mejor, que se vea mas dinamico asi especificamos correctamente cada punto

 **LauraMishifu**  **lost** 13:29
A coño, ahora lo he leído, que como estaba en clase si sacaba el móvil Antonio igual me decapitaba

 **ATSUSHINICA** 13:32
jajajaj okey, estaba modificando el documento algunas palabras o separaciones para que se vea más estetico, termino el de actividad, documentar, y me pongo con el main

 **LauraMishifu**  **lost** 14:05
Okey

 **ATSUSHINICA** 17:57
ya acabe con el diagrama de actividad
esta completamente documentado
nose si a ti te queda alguna cosilla mas

 **LauraMishifu**  **lost** 17:58
Yo que yo sepa no tengo nada más he comprobado también el diagrama de clases está mañana, y que yo haya visto no tenía nada más por cambiar

 **ATSUSHINICA** 18:14
vale
me puedes pasar el plantuml del diagrama de clases
porfis
lo has puesto en el main

 **LauraMishifu**  **lost** 18:18
Está en el main

 **ATSUSHINICA** 18:18
lo acabo de ver jsjsjs

 **LauraMishifu**  **lost** 18:18
JAJSJAJSA

 **ATSUSHINICA**

lo nas puesto en el main

 **LauraMishifu**  18:18

Está en el main

 **ATSUSHINICA** 18:18

lo acabo de ver jsjsjs

 **LauraMishifu**  18:18

JAJSAJSA
Sisi

 **ATSUSHINICA** 18:39

ya acabe la documentacion , retoque varias cosas
pero esta listo
acabo con el main y lo mando

 **LauraMishifu**  18:40

Me parece perfecto

 **LauraMishifu**  22:33

Has terminado el main?

 **LauraMishifu**  23:16

Estás vivo?

 **ATSUSHINICA** 23:17

estoy en ello
te voy a pedir un favor

 **LauraMishifu**  23:17

Dígamelo

 **ATSUSHINICA** 23:17

puedes colocar debajo de el documento un apartado de conversacion y reparto de tareas?
es que no me va a dar tiempo

 **LauraMishifu**  23:18

Okey

Reparto de Tareas

Diagrama de casos de uso → Alejandro.

Creación del diagrama de clases → Alejandro.

Modificaciones del diagrama de clases → Laura.

Diagrama de actividad → Alejandro.

Creación del diagrama de estados → Laura.

Modificación del diagrama de estados → Alejandro.

Creación de las clases y la programación → Laura.

Modificaciones de la programación para un perfecto funcionamiento → Alejandro.

Usuarios predefinidos del sistema

El sistema incluye una serie de usuarios registrados por defecto para poder acceder desde la primera ejecución. Cada usuario tiene asignado un rol que determina a qué menú accede y qué funcionalidades puede utilizar.

Como médicos se encuentran dr.garcia con contraseña garcia1234 y dr.martinez con contraseña martinez1234. Como personal administrativo están admin.lopez con contraseña lopez1234 y admin.perez con contraseña perez1234. Como personal de facturación se encuentran fact.ruiz con contraseña ruiz1234 y fact.torres con contraseña torres1234.

Estos usuarios se cargan automáticamente cada vez que arranca el sistema mediante el método inicializarUsuarios(). En un sistema real esta información estaría gestionada por una base de datos, pero dado que en este curso aún no se ha trabajado con ellas, se ha optado por almacenarlos directamente en el código.